

COMP3018: Report (Literature Review & Programming Project)

Cognitive Robotics

Contents

1- Task (3) Literature Review	1
1.1. Introduction	1
1.2. Literature Review	2
1.3. Applications	3
1.3.1 Therapeutic and Emotional Support	3
1.3.2 Medication Adherence and Daily Living Support	3
1.3.3 Physical Rehabilitation and Mobility	4
1.4. Discussion	4
1.4.1 Challenges	4
1.4.2 Ethical Implications	5
1.5. Conclusion	5
1.6. Task-3 References	6
1.6.1 Papers and Journal Articles	6
2- Task (4) Novel Programming Project (Adaptiveness in Assistive Robotics)	8
2.1. Introduction (10%; Salman)	8
2.2. Background (10%; Alfie)	8
2.3. Methods & Setup (35%; Alfie)	9
2.3.1 System Architecture	9
2.3.2 Input Layer: Multimodal and Behavioural Signals	9
2.3.3 Process Layer: Multi-Signal State Inference	13
2.3.4 Generate Layer: Dynamic Prompt Construction	16
2.3.5 Output Layer: Aligned Multimodal Response	17
2.3.6 Adaptation Self-Evaluation and Session Persistence	17
2.4. Outcome & System Analysis (30%; Salman)	17
2.5. Conclusion (10%; Alfie)	17
2.6 Task-4 References (5%)	17
3. Appendix	18
3.1 Video Demo	18
3.2 AI Declaration	19
3.3 Novel Python Project: gaze22.1.py	20

1- Task (3) Literature Review

1.1. Introduction

Ageing populations and a shrinking care workforce positioned **assistive robotics** (*human-supportive robots within physical, cognitive, and social realms of impairment affecting daily-living activities*); a prominent technological response to a widening care gap. The field spans diverse subfields; this essay however focuses

on *socially* assistive robotics (*SAR*), an assistive-robot subfield wherein the robot “focuses on helping human users through social rather than physical interaction” (Tapus, Matarić and Scassellati, 2007, Abstract), as here most active research and ethical tension converge.

Robots now administer medication reminders, facilitate rehabilitation exercises, and therapeutically companionate in clinical and domestic settings: reduced caregiver burden, improved patient outcomes wherein residents became “more active and more communicative, both with each other and their caregivers” (Wada and Shibata, 2007, p. 973).

However, most-current assistive robots operate at what Sciutti et al. (2023, p. 160) call the social layer: they react to immediate stimuli but lack the cognitive depth to anticipate user needs, remember past interactions, or reason about their own performance. Sciutti et al. argue that effective assistive robots should be *cognitive*: capable of “flexible, context-sensitive action, knowing what they are doing and why they are doing it.” Vernon, Metta and Sandini (2007, p. 151) formalise this requirement via a “virtuous cycle that is embedded in an ongoing process of action and perception” (the agent anticipates → learns → adapts to achieve autonomy). This essay contends that assistive robotics should graduate from reactive social behaviour to cognitive capability (intelligence deployed *over* the social layer) for sustained, personalised support. The sections as follows survey the theoretical foundations and the resulting applications, alongside the challenges and future directions.

1.2. Literature Review

Cognitive robotics: defined by Sandini, Sciutti and Vernon (2021, Overview), lies at the intersection of Robotics, Artificial Intelligence, and Cognitive and Biological Sciences, integrating “sensorimotor skills, knowledge representation and reasoning, and social interaction.” This interdisciplinary grounding distinguishes it from conventional robotics (*treats the robot as purely engineered*) and from social robotics (*addresses interaction behaviour without necessarily modelling cognitive processes*). The distinction is consequential: a robot that smiles when a patient smiles is social; a robot that infers *why*, and adapts accordingly, is *cognitive*.

42 catalogued definitions of cognition (euCognition) share a common thread: anticipation, learning, and adaptation, intersected with perception and action to create autonomy (Fig. 1). This cycle provides an architectural checklist for assistive robots: a system that cannot direct its gaze toward relevant stimuli whilst suppressing irrelevant ones (*selective attention*), anticipate the outcome of its actions (*i.e. prospection*), learn from past interactions (*memory*), or adapt its strategy when performance declines (*metacognition*) is, per this framework, not yet cognitive. Kotseruba and Tsotsos’s (2020, Abstract) survey of 84 cognitive architectures organises them along precisely these abilities, thus corroborating the checklist’s standing in the field. Sciutti et al. (2023, p. 160) further specify that cognitive robots should “reason about their actions and modify their behavior to improve their effectiveness”; a capacity termed *theory of mind*, wherein the agent infers another’s hidden mental state.

Furthermore, memory is not treated as a single uniform store (monolithic); Laird (2012, Chapter 9) distinguishes *episodic memory* (*records of specific past experiences and their contextual outcomes*) from *semantic memory* (*general knowledge about the world, including spatial relationships and factual constraints*) in implementational terms: episodic memory is “what you ‘remember’” whilst semantic memory is “what you ‘know’”. For example, assistive-medication robots need episodic memory to recall that a user refused medication after a restless night, and semantic memory to know certain drugs cannot also be administered.

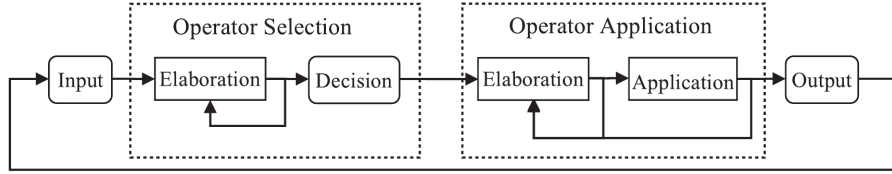


Figure 4.7
The Soar processing cycle.

Figure 1: Soar’s processing cycle, reproduced from Laird (2012, fig. 4.7, p. 79), as a concrete instantiation of Vernon, Metta and Sandini’s (2007) cognition cycle. Laird specifies that the cycle “consists of four phases: Input, Operator Selection, Operator Application, and Output” (Laird, 2012, p. 79), wherein Input maps to perception, Operator Selection draws on episodic and semantic memory to anticipate consequences (i.e. *prospection*), Operator Application executes the chosen behaviour through parallel rule-firing waves, and Output returns the agent to the environment whilst the loop hands control back to Input for continual adaptation. This architecture exposes the deficit of reactive systems: PARO, for instance, implements only Input → Output, lacking the substate-driven Operator Selection wherein deliberation over latent user states could occur. Furthermore, Laird argues that “the most significant effect of chunking is that it eliminates processing in substates for situations similar to ones experienced in the past” (Laird, 2012, p. 164), wherein repeated deliberations compile into procedural rules; a mechanism distinct from POMDP belief-space approximation, yet complementary thereto in addressing real-time embodied operation under care-time constraints.

1.3. Applications

1.3.1 Therapeutic and Emotional Support

The PARO therapeutic seal robot is among the most-widely deployed platforms within socially assistive robotics (Tapus, Matarić and Scassellati, 2007, .pdf-p. 1). Wada and Shibata (2007, p. 973) demonstrate that PARO improves mood in patients with dementia, utilising tactile and auditory inputs. Clinical trials report that urinary stress indicators “significantly improved” after PARO’s introduction (Wada and Shibata, 2007, p. 978, Table II), therefore the platform has been adopted in care homes.

Notwithstanding these benefits PARO operates at the reactive layer, possessing no theory of mind (cannot infer *why* a patient is agitated (loneliness, pain, confusion) nor episodic memory of *what* calmed the patient previously). A cognitively-equipped therapeutic robot, in contrast, would anticipate mood shifts (*prospection*), recall what calmed the patient (episodic memory), and adapt its strategy (metacognition). PARO’s effectiveness plateaus precisely because it cannot personalise; the gap is therefore clinically consequential. The architectural cost is precise: without episodic memory, the agent’s perception is “limited both spatially and temporally—that is, to the here and now” (Laird, 2012, p. 233). Fong, Nourbakhsh and Dautenhahn (2003, p. 145) formalise this gap via Breazeal’s taxonomy: PARO occupies the “social interface” level (human-like cues but “shallow models of social cognition”), whereas Sciutti et al.’s (2023, p. 160) vision of robots “knowing what they are doing and why” demands the *socially intelligent* level.

1.3.2 Medication Adherence and Daily Living Support

Medication non-adherence imposes substantial costs on healthcare systems, and elderly patients with polypharmacy regimens are particularly vulnerable to missed or incorrect doses. Robots in this domain face a different challenge from therapeutic companionship: trust and cognitive load are latent variables, inferred only from noisy behavioural proxies. Lee and See (2004, .pdf-p. 6) define trust as “the attitude that an agent will help achieve an individual’s goals in a situation characterized by uncertainty and vulnerability”; a definition foregrounding the unobservable nature that necessitates probabilistic modelling. A user may comply with a medication prompt despite low trust (e.g. time pressure), or indeed refuse despite high trust (e.g. task complexity), and thus the observation alone cannot reliably disambiguate the underlying state (Kaelbling, Littman and Cassandra, 1998, p. 105, 3. *Partial observability*).

The Partially Observable Markov Decision Process (POMDP) provides formal machinery for this uncertainty. Chen et al. (2020, p. 6) demonstrate a Trust-POMDP wherein the robot maintains a belief distribution over trust and selects actions that maximise long-term collaboration, showing belief-space planning outperforms fixed strategies. Garcez and Lamb (2023, *.pdf*-p. 1) identify the neuro-symbolic paradigm as the ‘third wave’ of AI, wherein neural subsystems (LLMs) handle perception whilst symbolic subsystems (e.g. POMDPs) govern temporal reasoning. Nikolaidis, Hsu and Srinivasa (2017, p. 625) provide empirical corroboration: in a collaborative task ($n = 69$), robots utilising mutual adaptation via a Mixed Observability MDP (modelling human adaptability as a latent variable) were rated significantly more trustworthy than fixed-policy alternatives ($U = 180$, $p = 0.048$). This aligns with Hancock et al.’s (2011, p. 522) finding that performance is the strongest trust predictor.

1.3.3 Physical Rehabilitation and Mobility

Robotic exoskeletons and assistive manipulators for stroke recovery and mobility support need to adapt not only to the patient’s physical state (joint angles and muscle activation patterns) but also to their psychological state: motivation, frustration, and fatigue determine whether a patient perseveres or disengages.

Embodied cognition becomes essential. Brooks (1991, Introduction) argues that intelligence emerges from physical interaction with the environment rather than abstract representation, whereas Fong, Nourbakhsh and Dautenhahn (2003, p. 149) operationalise this as “perturbatory coupling”: the more channels of mutual influence, the more embodied the system. A rehabilitation robot therefore occupies a uniquely cognitive niche, sensing the patient’s body, reasoning about current capabilities, and adapting appropriately. A purely language-based or screen-based interface cannot achieve this; Mataric et al. (2007, *.pdf*-p. 7) confirm: stroke survivors engaged more with embodied robots than screen-based alternatives. Tapus, Tăpuş and Mataric (2008, Abstract) show that embodiment alone is insufficient: adaptive personality matching (adjusting interaction distance and speed to the user’s traits) further improved task performance, suggesting rehabilitation robots require physical presence and cognitive adaptation.

1.4. Discussion

1.4.1 Challenges

Tapus, Mataric and Scassellati (2007, *.pdf*-p. 6) projected that by 2012 SAR systems would demonstrate “marked improvement in learning/training/recovery of the user”; yet PARO, the most-deployed platform nearly twenty years later, *still* cannot remember yesterday’s session. Three challenges explain this. Firstly, computational intractability: solving *POMDPs* exactly is PSPACE-complete (Papadimitriou and Tsitsiklis, 1987, p. 448), and the belief simplex grows exponentially with state-space dimensionality. Whilst approximate solvers such as point-based value iteration (Pineau, Gordon and Thrun, 2003, p. 1025) and online Monte-Carlo tree search (Silver and Veness, 2010, p. 1) mitigate this, real-time cognitive processing within embodied systems remains an open challenge, particularly when multiple unobserved variables (trust, load, emotion) require tracking simultaneously.

Secondly, the measurement problem: trust, cognitive load, and emotional state are not directly observable; observations thereof are noisy proxies at best. Hancock et al.’s (2011, p. 522) meta-analysis of 29 studies finds that even the strongest correlates of trust explain only moderate variance, whilst Broadbent, Stafford and MacDonald (2009, Abstract) note that acceptance depends on matching robot behaviour to user expectations, not trust alone. Desai et al. (2013) demonstrate trust dynamics are non-linear, building slowly through consistent performance but degrading rapidly after errors; and thus a single misclassified observation can cascade into wrong actions. Nikolaidis, Hsu and Srinivasa (2017, p. 626), however, demonstrate that mutual adaptation partially mitigates this fragility: when the robot models human adaptability as a latent variable, trust persists even during strategy disagreements, suggesting the variance Hancock et al. report may stem from studies that treat the human as a static rather than co-adaptive partner.

Finally, adaptation without exploitation: a robot that runs inference on cognitive load could time its medication requests to coincide with periods of high vulnerability, maximising compliance at the expense of user autonomy. The POMDP reward function should therefore encode ethical constraints alongside clinicians’

objectives.

1.4.2 Ethical Implications

Assistive robots in intimate care spaces (bedrooms, bathrooms, rehabilitation clinics) continuously collect sensitive behavioural data. Facial expressions, vocal patterns, and movement trajectories constitute biometric data, yet regulatory frameworks have not kept pace with deployment. Wachter, Mittelstadt and Floridi (2017, Abstract) argue that even the General Data Protection Regulation provides no enforceable “right to explanation” of automated decisions; a gap particularly concerning in healthcare wherein recommendations directly affect patient wellbeing.

Moreover, over-reliance on assistive robots risks eroding functional independence. If a robot anticipates and pre-emptively needs via prospection, the user may disengage from self-directed activity, contradicting the assistive mandate. Sharkey (2014, *.pdf*-p. 6) frames this via Nordenfelt’s ‘Dignity of Identity’: “a robot that dealt impersonally with an older person, without knowing or using their name or their preferences would also be likely to negatively affect their feelings of dignity.” This implies that only cognitively-equipped robots (with episodic memory of individuals) can avoid dignity violations; reactive systems such as PARO, regardless of their therapeutic benefits (Wada and Shibata, 2007, p. 973), risk infantilisation precisely because they cannot personalise. The responsibility gap compounds this further: when a care robot administers incorrect medication, liability falls ambiguously between manufacturer, deployer, and clinician.

The ethical watchword is therefore proactive regulation: design-stage ethics anticipating failure modes, not reactive patchwork after harm. Per the embodied cognition thesis, if intelligence indeed requires a body, and that body enters the most intimate spaces of vulnerable persons, then the ethical stakes of assistive cognitive robotics are uniquely high.

1.5. Conclusion

Assistive robotics stands at an inflection point. Current systems deliver measurable benefits within narrow envelopes, yet their reactive architectures limit sustained, personalised effectiveness. The Vernon, Metta and Sandini (2007) cognition cycle provides the architectural blueprint for graduating beyond this plateau: assistive robots that anticipate (prospection), remember (episodic and semantic memory), reason about others’ mental states (theory of mind), and monitor their own performance (metacognition) would mark a qualitative advance over current systems.

The neuro-symbolic paradigm offers a viable path toward this vision, as the Trust-POMDP framework attests (Chen et al., 2020). Sciutti et al. (2023, p. 161) independently identify the integration of learning with model-based approaches as cognitive robotics’ most-prominent trajectory; that this converges with Garcez and Lamb’s (2023, *.pdf*-p. 1) ‘third wave’ thesis suggests the direction is robust rather than parochial. Sharkey and Sharkey (2012, *.pdf*-p. 1) caution that such systems risk replacing rather than supplementing human-care; the field should therefore pursue cognitive capability and ethical governance in concert, lest the technology displace these very carers it was meant to supplement. Figure~2 visualises this trajectory. The ultimate test, per the embodied cognition thesis, is a robot that can: sense → remember → anticipate → adapt within the physical world, whilst respecting the autonomy, and dignity, of the persons it serves.

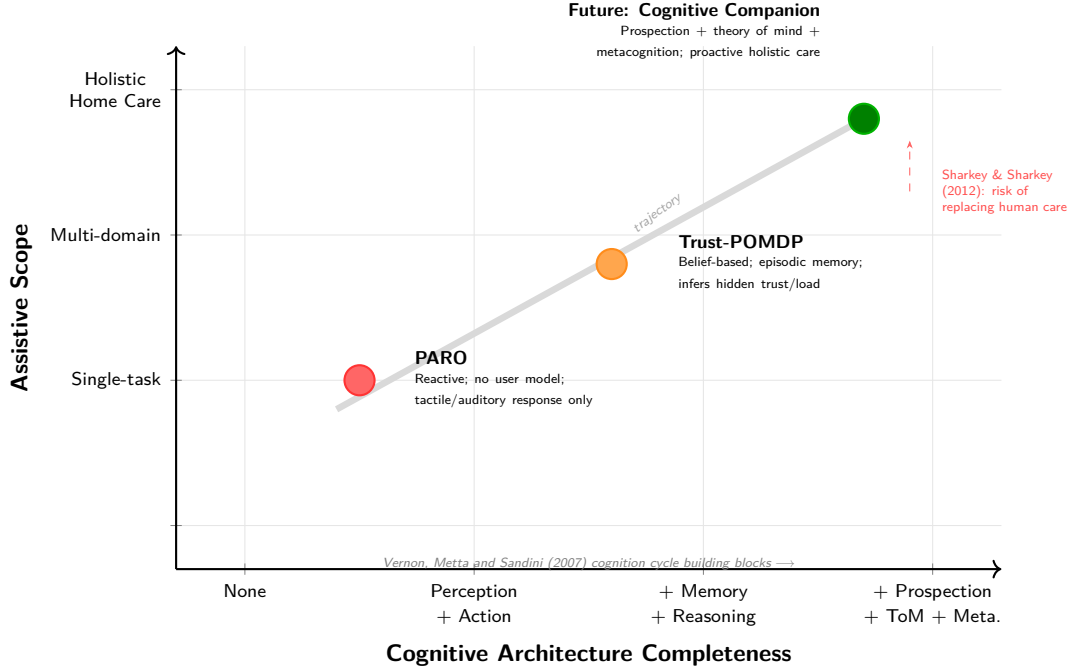


Figure 2: Trajectory of assistive robotics from reactive single-task systems (PARO) through adaptive belief-based architectures (Trust-POMDP) toward cognitively autonomous home-dwelling companions. Expanding assistive scope without expanding cognitive capability is insufficient; the diagonal trajectory requires both.

1.6. Task-3 References

1.6.1 Papers and Journal Articles

- Broadbent, E., Stafford, R. and MacDonald, B. (2009) ‘Acceptance of Healthcare Robots for the Older Population: Review and Future Directions’, *International Journal of Social Robotics*, 1(4), pp. 319-330. Available at: https://www.researchgate.net/publication/220397395_Acceptance_of_Healthcare_Robots_for_the_Older_Population (Accessed: 25 March 2026).
- Brooks, R. A. (1991) ‘Intelligence without representation’, *Artificial Intelligence*, 47(1-3), pp. 139-159. Available at: <https://people.csail.mit.edu/brooks/papers/representation.pdf> (Accessed: 24 March 2026).
- Chen, M., Nikolaidis, S., Soh, H., Hsu, D. and Srinivasa, S. (2020) ‘Trust-Aware Decision Making for Human-Robot Collaboration: Model Learning and Planning’, *ACM Transactions on Human-Robot Interaction*, 9(2), pp. 1-23. Available at: https://personalrobotics.cs.washington.edu/publications/ch_en2019trust.pdf (Accessed: 15 March 2026).
- Desai, M., Kaniarasu, P., Medvedev, M., Steinfeld, A. and Yanco, H. (2013) ‘Impact of robot failures and feedback on real-time trust’, *Journal of Human-Robot Interaction*, 2(1), pp. 251-275. Available at: <https://ieeexplore.ieee.org/document/6483596> (Accessed: 20 March 2026).
- Fong, T., Nourbakhsh, I. and Dautenhahn, K. (2003) ‘A survey of socially interactive robots’, *Robotics and Autonomous Systems*, 42(3-4), pp. 143-166. Available at: <https://www.cs.cmu.edu/~illah/PAPERS/socialroboticssurvey.pdf> (Accessed: 18 March 2026).
- Garcez, A. d’A. and Lamb, L. C. (2023) ‘Neurosymbolic AI: The 3rd Wave’, *Artificial Intelligence Review*, 56, pp. 12387-12406. Available at: <https://link.springer.com/article/10.1007/s10462-023-10448-w> (Accessed: 20 March 2026).
- Hancock, P. A., Billings, D. R., Schaefer, K. E., Chen, J. Y. C., de Visser, E. J. and Parasuraman, R.

- (2011) ‘A meta-analysis of factors affecting trust in human-robot interaction’, *Human Factors*, 53(5), pp. 517-527. Available at: <https://journals.sagepub.com/doi/10.1177/0018720811417254> (Accessed: 15 March 2026).
- Kaelbling, L. P., Littman, M. L. and Cassandra, A. R. (1998) ‘Planning and acting in partially observable stochastic domains’, *Artificial Intelligence*, 101(1-2), pp. 99-134. Available at: <https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf> (Accessed: 13 March 2026).
 - Kotseruba, I. and Tsotsos, J. K. (2020) ‘40 years of cognitive architectures: core cognitive abilities and practical applications’, *Artificial Intelligence Review*, 53(1), pp. 17-94. Available at: <https://link.springer.com/article/10.1007/s10462-018-9646-y> (Accessed: 3 May 2026).
 - Laird, J. E. (2012) *The Soar Cognitive Architecture*. Cambridge, MA: MIT Press.
 - Lee, J. D. and See, K. A. (2004) ‘Trust in automation: Designing for appropriate reliance’, *Human Factors*, 46(1), pp. 50-80. Available at: https://journals.sagepub.com/doi/10.1518/hfes.46.1.50_30392 (Accessed: 15 March 2026).
 - Matarić, M. J., Eriksson, J., Feil-Seifer, D. J. and Winstein, C. J. (2007) ‘Socially assistive robotics for post-stroke rehabilitation’, *Journal of NeuroEngineering and Rehabilitation*, 4(5), pp. 1-9. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC1821334/> (Accessed: 25 March 2026).
 - Nikolaidis, S., Hsu, D. and Srinivasa, S. (2017) ‘Human-robot mutual adaptation in collaborative tasks: Models and experiments’, *The International Journal of Robotics Research*, 36(5-7), pp. 618-634. Available at: <https://journals.sagepub.com/doi/10.1177/0278364917690593> (Accessed: 20 March 2026).
 - Papadimitriou, C. H. and Tsitsiklis, J. N. (1987) ‘The complexity of Markov decision processes’, *Mathematics of Operations Research*, 12(3), pp. 441-450. Available at: <https://web.mit.edu/jnt/www/Papers/J016-87-mdp-complexity.pdf> (Accessed: 13 March 2026).
 - Pineau, J., Gordon, G. and Thrun, S. (2003) ‘Point-based value iteration: An anytime algorithm for POMDPs’, in *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI-03)*, pp. 1025-1030. Available at: <http://www.cs.cmu.edu/~ggordon/jpineau-ggordon-thrun.ijcai03.pdf> (Accessed: 24 March 2026).
 - Sandini, G., Sciutti, A. and Vernon, D. (2021) ‘Cognitive Robotics’, in *Encyclopedia of Robotics*. Berlin: Springer-Verlag. Available at: http://www.vernon.eu/publications/2021_Sandini_et_al.pdf (Accessed: 3 May 2026).
 - Sciutti, A., Beetz, M., Inamura, T., Korsah, A., Oh, J., Sandini, G., Shimoda, S. and Vernon, D. (2023) ‘The Present and the Future of Cognitive Robotics’, *IEEE Robotics & Automation Magazine*, 30(3), pp. 160-163. Available at: <https://ieeexplore-ieee-org.plymouth.idm.oclc.org/document/10255092> (Accessed: 18 March 2026).
 - Sharkey, A. (2014) ‘Robots and human dignity: A consideration of the effects of robot care on the dignity of older people’, *Ethics and Information Technology*, 16(1), pp. 63-75. Available at: <https://philarchive.org/rec/SHARAH-2> (Accessed: 22 March 2026).
 - Sharkey, A. and Sharkey, N. (2012) ‘Granny and the robots: ethical issues in robot care for the elderly’, *Ethics and Information Technology*, 14(1), pp. 27-40. Available at: <https://philarchive.org/rec/SHAGAT> (Accessed: 22 March 2026).
 - Silver, D. and Veness, J. (2010) ‘Monte-Carlo planning in large POMDPs’, in *Advances in Neural Information Processing Systems (NeurIPS 23)*, pp. 2164-2172. Available at: <https://proceedings.neurips.cc/paper/2010/file/Paper.pdf> (Accessed: 24 March 2026).
 - Tapus, A., Matarić, M. J. and Scassellati, B. (2007) ‘Socially assistive robotics [Grand Challenges of Robotics]’, *IEEE Robotics & Automation Magazine*, 14(1), pp. 35-42. Available at: <https://sczlab.yale.edu/sites/default/files/files/Tapus-RAM2007.pdf> (Accessed: 25 March 2026).

- Tapus, A., Țăpuș, C. and Matarić, M. J. (2008) ‘User-robot personality matching and assistive robot behavior adaptation for post-stroke rehabilitation therapy’, *Intelligent Service Robotics*, 1(2), pp. 169-183. Available at: <https://hal.science/hal-00770108/document> (Accessed: 26 March 2026).
- Vernon, D., Metta, G. and Sandini, G. (2007) ‘A Survey of Artificial Cognitive Systems: Implications for the Autonomous Development of Mental Capabilities in Computational Agents’, *IEEE Transactions on Evolutionary Computation*, 11(2), pp. 151-180. Available at: https://www.robotcub.org/misc/papers/07_Vernon_Metta_Sandini_IEEE.pdf (Accessed: 13 March 2026).
- Wachter, S., Mittelstadt, B. and Floridi, L. (2017) ‘Why a Right to Explanation of Automated Decision-Making Does Not Exist in the General Data Protection Regulation’, *International Data Privacy Law*, 7(2), pp. 76-99. Available at: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2903469 (Accessed: 22 March 2026).
- Wada, K. and Shibata, T. (2007) ‘Living with seal robots: its sociopsychological and physiological influences on the elderly at a care house’, *IEEE Transactions on Robotics*, 23(5), pp. 972-980. Available at: <https://ieeexplore.ieee.org/document/4339551> (Accessed: 18 March 2026).

2- Task (4) Novel Programming Project (Adaptiveness in Assistive Robotics)

2.1. Introduction (10%; Salman)

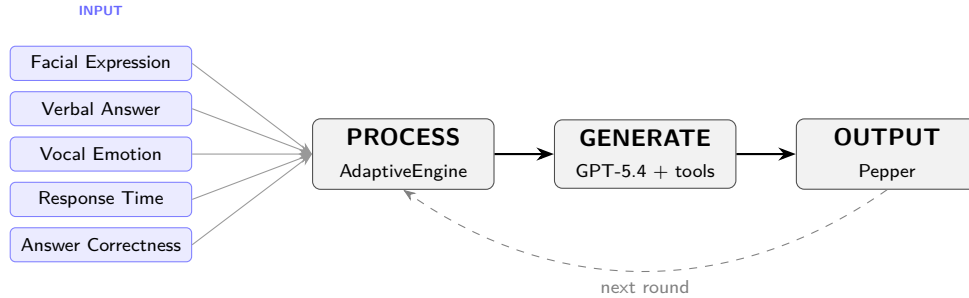


Figure 3: GAZE system architecture. Five live input signals feed the AdaptiveEngine (the symbolic reasoning layer), wherein a context block is built and passed to GPT-5.4 function-calling; a separate deterministic GPT-4.1 call handles answer verification (i.e. the referee, not the host). The generated response delivers concurrently via Pepper’s speech, gesture, and LED subsystems, whilst the dashboard pulls frames from Pepper’s ALVideoDevice socket stream rather than still photos. The loop circulates to the next round, now adapted.

2.2. Background (10%; Alfie)

GAZE sits within socially assistive robotics (*the deployment of robots to support users through social interaction rather than physical contact*) and affective computing, which Picard (1997, p. 1, Abstract) founds as “computing that relates to, arises from, or influences emotions”; Tapus, Matarić and Scassellati (2007, .pdf- p. 1) define this sub-field as systems that “assist users through social interaction.” Most-current platforms react to a single input signal, suffering the single-signal problem: a facial-expression classifier misreads resting faces as displeasure; a response-time metric mistakes deliberation for disengagement. Calvo and D’Mello (2010, p. 28) identify “the inherent challenges with unisensory affect detection”; Poria et al. (2017, p. 99) report that multimodal systems were “consistently (85% of systems) more accurate than their best unimodal counterparts, with an average improvement of 9.83%.” Spezialetti, Placidi and Rossi (2020, pp. 1-2, Introduction) substantiate this within HRI via reviewings of recognition systems across facial, vocal, and physiological channels. This demands multi-signal fusion (the system weighs complementary channels together instead of trusting any one alone).

GAZE’s core contribution: multi-signal emotional inference; facial expression (CNN, Workshop 10), vocal emotion (MLP, Workshop 8), speech volume/RMS, response time, answer correctness, and answer text fuse with derived temporal signals into a single-inferred user-state. The implementation is face-primary: voice nudges only when the face is Neutral and the MLP clears 0.9 confidence (**fearful** excluded as the silence-attractor). GPT-5.4 generates dialogue whilst the symbolic AdaptiveEngine governs state inference, grounding output in Pepper’s affordances (Ahn et al., 2022, *.pdf*-p. 1, Abstract); a GPT-4.1 verifier handles answer-checking. Smedegaard (2019, p. 414, Experiential novelty) warns engagement reflects novelty rather than sustained interest. GAZE’s adaptive engine therefore aims to sustain engagement beyond the novelty phase by adaptively (dynamically) adjusting difficulty, switching games, and drafting adapted questions based on inferred user-state.

2.3. Methods & Setup (35%; Alfie)

2.3.1 System Architecture

GAZE operates as a conversational loop rather than a rigid question-answer cycle, wherein each turn fans five live signals into the AdaptiveEngine, GPT-5.4 generates an adapted response via function calling, and Pepper executes speech, gesture, and LED concurrently.

This function-calling architecture is neuro-symbolic: GPT-5.4 controls dialogue and decision-making, whilst the AdaptiveEngine and game logic are callable tools. This aligns with Garcez and Lamb’s (2023, *.pdf*-p. 1) *third wave* paradigm, wherein neural and symbolic components share a structured interface (cf. Ahn et al., 2022, *.pdf*-p. 1).

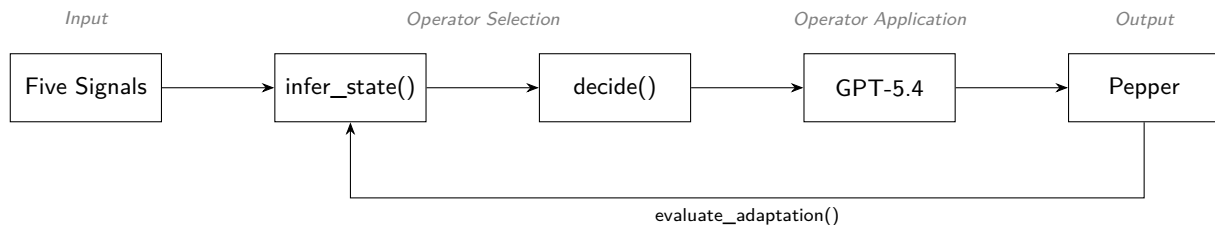


Figure 4: GAZE’s per-turn cycle mapped onto Laird’s (2012, fig. 4.7, p. 79) Soar phases: five signals feed Input, `infer_state()` and `decide()` perform Operator Selection, *gpt-5.4* handles Operator Application, whilst *Pepper* executes Output. Unlike Soar’s implicit chunking, GAZE instead closes the loop via an explicit `evaluate_adaptation()` step that carries over to the next turn.

2.3.2 Input Layer: Multimodal and Behavioural Signals

1- Facial Expression (vision-based). A pre-trained CNN (Workshop 10) classifies the user’s expression into seven Ekman categories, building upon Ekman and Friesen (1971, p. 128), whose cross-cultural results show that “particular facial behaviors are universally associated with particular emotions,” finding that even preliterate (not written language) cultures with “minimal opportunity to have learned to recognize uniquely Western facial expressions” identified the same six emotions; this taxonomy remains “still the most popular perspective for FER” (Li and Deng, 2020, p. 1). Known limitations (cultural bias, resting-face misclassification) are mitigated by combining facial evidence with behavioural signals rather than allowing the CNN to decide user state alone.

2- Verbal Answer (speech-based). Pepper records through all four microphones (the [1,1,1,1] mask), polling max energy across all mics hence catching off-axis speakers a front-only poll would miss. Recording terminates after 1.2 seconds of post-speech silence or at the adaptive hard ceiling. After SFTP, `force_mono_16k_wav()` canonicalises to mono 16 kHz by loudest-channel RMS. Whisper (**whisper-1**), whose models “approach [human] accuracy and robustness” (Radford et al., 2023, Abstract), is utilised as a delivery-aware sensor: `verbose_json` exposes `no_speech_prob`, `avg_logprob`, and `compression_ratio`, encoding

how the user spoke alongside *what* they said. Barge-in was deliberately omitted owing to NAOqi's imperfect acoustic echo cancellation (Pepper's own voice would fire false interrupts).

Listing 1: `nao_record`: calibrated-threshold silence detection polling the max energy across all four Pepper mics, with firmware fallback. Recording uses the `[1,1,1,1]` channel mask; `force_mono_16k_wav()` downstream picks the Loudest channel by RMS so Whisper, Silero-VAD, the WS-08 MLP, and `measure_volume()` all see one canonical mono 16 kHz layout.

```
1 def nao_record(ssh, energy_threshold: int = DEFAULT_ENERGY_THRESHOLD,
2               record_max_secs: float = RECORD_MAX_SECS,
3               silence_secs: float = SILENCE_DURATION):
4     """Record audio on Pepper with dynamic silence detection.
5     - poll the max energy across all four mics to stop recording early if silence detected
6     - calibrated energy threshold to avoid false positives from ambient noise
7     - if getFrontMicEnergy is unsupported (e.g. older firmware), fall back to a safe
      ↪ fixed-duration recording to ensure the demo still works, albeit without silence
      ↪ detection
8     """
9
10    nao_run(ssh, f"""
11 from naoqi import ALProxy
12 import time
13
14 rec = ALProxy("ALAudioRecorder", "127.0.0.1", 9559)
15
16 rec.stopMicrophonesRecording()
17 rec.startMicrophonesRecording("{REMOTE_WAV}", "wav", 16000, [1, 1, 1, 1])
18
19 try:
20     audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
21
22     speech_detected = False
23     silence_start = None
24     start = time.time()
25     threshold = {energy_threshold}
26
27     while True:
28         elapsed = time.time() - start
29
30         # hard ceiling; never exceed max duration
31         if elapsed >= {record_max_secs}:
32             break
33
34         # poll all four mics and take the MAX; a user stood to either side of Pepper registers on the
35         ↪ side mics but not the front, so front-only polling misses them and the silence-detector
36         ↪ stops recording mid-utterance
37         try:
38             energy = max(
39                 audio.getFrontMicEnergy(),
40                 audio.getLeftMicEnergy(),
41                 audio.getRightMicEnergy(),
42                 audio.getRearMicEnergy(),
43             )
44         except Exception:
45             energy = audio.getFrontMicEnergy() # firmware fallback
46
47         if elapsed < {RECORD_MIN_SECS}:
48             # minimum recording period
49             if energy > threshold:
```

```

48         speech_detected = True
49         time.sleep({SILENCE_POLL_SECS})
50         continue
51
52     if energy > threshold:
53         speech_detected = True
54         silence_start = None
55     else:
56         if speech_detected and silence_start is None:
57             silence_start = time.time()
58         if speech_detected and silence_start is not None:
59             if (time.time() - silence_start) >= {silence_secs}:
60                 break
61
62     time.sleep({SILENCE_POLL_SECS})
63
64 except Exception as e:
65     # firmware fallback; getFrontMicEnergy() unsupported on this Pepper
66     # fall back to a safe fixed-duration recording so the demo never breaks
67     print(" [Silence detection failed: " + str(e) + "] Falling back to fixed-duration recording")
68     time.sleep({record_max_secs})
69
70 rec.stopMicrophonesRecording()
71 """
72 sftp = ssh.open_sftp()
73 sftp.get(REMOTE_WAV, LOCAL_WAV)
74 sftp.close()

```

Whisper hallucinates on near-silent audio, emitting training-set tics (CJK gibberish, prompt-primed repetition); the latter is NLG *degeneration* wherein models get “stuck in repetitive loops” (Ji et al., 2023, p. 3). Transcription is therefore gated by a five-layer defence chain (Listing~2).

Listing 2: transcribe: five-layer defence chain against Whisper hallucination on near-silent audio (gaze.py, lines 1532–1610).

```

1 def transcribe(bypass_wake_word: bool = False,
2               whisper_prompt: str = "User answers a quiz or chats with a companion robot.",
3               record_again=None,
4               max_hallucination_retries=None) -> str:
5     # Silero VAD -> wake-word -> Whisper -> hallucination blacklist
6     attempts_remaining = max_hallucination_retries
7     while True:
8         if INPUT_IS_LOCAL and not _local_speech_detected:
9             return ""
10
11         # Silero VAD hard gate; NAO + LOCAL
12         if not has_real_speech(LOCAL_WAV):
13             print(" Silero VAD found no speech; skipping Whisper.")
14             return ""
15
16         # Vosk wake-word gate; bypass for name prompt
17         if not bypass_wake_word and not has_wake_word(LOCAL_WAV):
18             print(" No wake-word detected; skipping Whisper.")
19             return ""
20
21     try:
22         with open(LOCAL_WAV, "rb") as fh:
23             resp = client.audio.transcriptions.create(
24                 model="whisper-1", #

```

```

25         file=fh, #
26         response_format="verbose_json", # get self-signals for hallucination detection
27         temperature=0.0, # zero randomness
28         prompt=whisper_prompt,
29         timeout=API_TIMEOUT,
30     )
31     text = (getattr(resp, "text", "") or "").strip()
32     print(f" Whisper raw text: {text!r}")
33
34     # Whisper self-signals from verbose_json
35     segments = getattr(resp, "segments", None) or []
36     suspected_hallucination = False
37     if segments:
38         no_speech_vals = [s.no_speech_prob for s in segments
39                           if getattr(s, "no_speech_prob", None) is not None]
40         logprob_vals = [s.avg_logprob for s in segments
41                         if getattr(s, "avg_logprob", None) is not None]
42         compression_vals = [s.compression_ratio for s in segments
43                             if getattr(s, "compression_ratio", None) is not None]
44         print(f" Whisper diagnostics: no_speech={no_speech_vals},
45               ↪ avg_logprob={logprob_vals}, compression={compression_vals}")
46         if no_speech_vals and max(no_speech_vals) > 0.6:
47             print(f" Whisper flagged silence (max no_speech_prob={max(no_speech_vals):.2f});
48                   ↪ dropping {text!r}")
49             return ""
50         if logprob_vals and min(logprob_vals) < -1.3:
51             print(f" Whisper low-confidence (min avg_logprob={min(logprob_vals):.2f});
52                   ↪ dropping {text!r}")
53             return ""
54         # repetition-loop hallucination; flag for retry path
55         if compression_vals and max(compression_vals) > 2.4:
56             print(f" Whisper repetition loop (max
57                   ↪ compression_ratio={max(compression_vals):.2f}); flagging {text!r} as
58                   ↪ hallucination")
59             suspected_hallucination = True
60
61     # normalised hallucination blacklist + Whisper-self-signal hallucinations
62     if suspected_hallucination or is_known_hallucination(text):
63         print(f" Filtered Whisper hallucination: {text!r}")
64         if record_again is not None and (attempts_remaining is None or attempts_remaining >
65             ↪ 0):
66             if attempts_remaining is not None: # whilst more retries remain
67                 attempts_remaining -= 1
68                 print(f" Disregarding hallucination; listening again (attempts left:
69                       ↪ {attempts_remaining}).")
70             else:
71                 print(f" Disregarding hallucination; listening again.")
72                 record_again()
73                 continue
74     return ""
75
76     # Strip leading "Pepper"/"Gaze" so handlers receive just the answer; \b blocks
77     ↪ "Pepperoni"/"Gazebo" false positives..
78     stripped = re.sub(r'(?i)~s*(pepper|gaze)\b[.,\s]*', '', text).strip()
79     return stripped
80 except Exception as e:
81     print(f" Whisper transcribe failed ({e}); returning empty")
82     return ""

```

3- Vocal Emotion (audio-based). The same WAV passes through a pre-trained MLP (Workshop 8) *before* transcription, classifying vocal state into four emotions (calm, happy, fearful, disgust) via MFCC, chroma, and mel-spectrogram features; El Ayadi, Kamel and Karray (2011, p. 578) identify MFCCs as “the most promising features” for speech-emotion recognition. This provides a second, independent modality; the two may disagree, wherein decision logic arbitrates. RAVDESS is acted-speech, and indeed Williams and Stevens (cited in El Ayadi et al., 2011, p. 573) found “acted emotions tend to be more exaggerated than real ones”; the WS-08 mean-pooling further loses “temporal information present in speech signals” (El Ayadi, Kamel and Karray, 2011, p. 574), hence fear’s tremor structure collapses indistinguishable from low-energy real-room speech. Voice is therefore engineered as a tie-breaker (Section 2.3.3), an architectural response to a documented limitation rather than a debugging afterthought.

Listing 3: `SpeechEmotionModel.extract_features`: MFCC + chroma + mel-spectrogram feature vector matching the WS-08 training pipeline, with peak-normalisation for quieter brain-injured users (`gaze.py`, lines 264–292).

```

1
2 @staticmethod # static because it's also used independently in classify_speech_emotion()
3     def extract_features(wav_path: str):
4         "Extract the same MFCC/chroma/mel feature vector used in WS-08 training."
5         with sf.SoundFile(wav_path) as sound_file:
6             audio = sound_file.read(dtype="float32")
7             sample_rate = sound_file.samplerate
8
9             if audio.ndim > 1:
10                 audio = audio.mean(axis=1)
11
12             # peak-normalise; helps quieter speakers
13             audio = librosa.util.normalize(audio)
14
15             n_fft = 2048
16             if len(audio) < n_fft:
17                 return None
18
19             stft = np.abs(librosa.stft(audio, n_fft=n_fft))
20
21             mfccs = np.mean(librosa.feature.mfcc(
22                 y=audio, sr=sample_rate, n_mfcc=40).T, axis=0).flatten()
23
24             chroma = np.mean(librosa.feature.chroma_stft(
25                 S=stft, sr=sample_rate).T, axis=0).flatten()
26
27             mel = np.mean(librosa.feature.melspectrogram(
28                 y=audio, sr=sample_rate).T, axis=0).flatten()
29
30             return np.concatenate([mfccs, chroma, mel])

```

4- Response Time (engagement-based). A Python timer measures elapsed time from question delivery to recording completion; the Whisper call occurs *after* the timer halts, isolating deliberation time from API latency.

5- Answer Correctness (task-based). GPT-4.1’s `check_game_answer` evaluates the transcribed answer at a deterministic temperature; the resultant binary correctness feeds the `AdaptiveEngine` and the derived rolling-correctness signal.

2.3.3 Process Layer: Multi-Signal State Inference

The `AdaptiveEngine`’s `infer_state()` classifies the user into one of five states (*Thriving*, *Comfortable*, *Struggling*, *Frustrated*, *Disengaged*) from five raw inputs supplemented by three derived temporal features (*rolling*

correctness over the last five rounds, consecutive-silence count, consecutive-wrong streak length). Crucially, classification is face-primary: facial expression, correctness, response time, silence, wrong-streaks, and volume carry the decision; vocal emotion fires only as a high-confidence tie-breaker when face is Neutral AND the MLP clears 0.9 AND the label is not **fearful** (the silence-attractor; see §2.3.2). Voice is thus retained as evidence but never allowed to override stronger behavioural readings, addressing the brittleness Desai et al. (2013) observe in single-signal systems. Thresholds were derived from pilot testing.

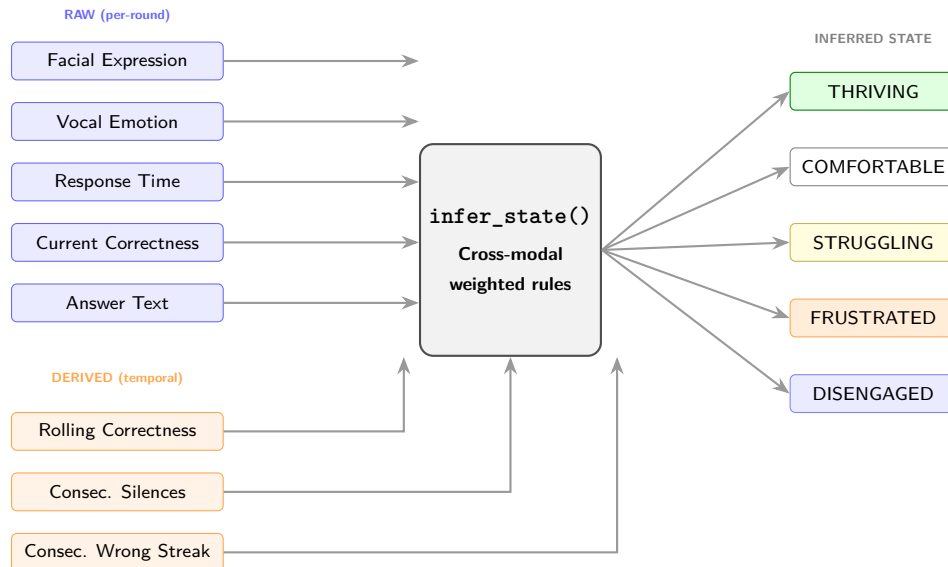


Figure 5: Multi-signal inference pipeline. Five raw inputs captured each round and three derived temporal signals computed from session history feed into `infer_state()`, which applies cross-modal rules (e.g. camera reads *Angry* but user answers fast and correctly → *Comfortable*) to classify the user into one of five states. Voice is treated as advisory rather than dominant: it nudges the state only when the face is Neutral AND the MLP’s confidence clears 0.9 AND the label is not **fearful** (the silence-attractor), hence a noisy vocal label can never override stronger visual or behavioural evidence; therein lies the multi-signal novelty.

Listing 4: Multi-signal inference rules (verbatim excerpt from `gaze.py infer_state()`; method-body indentation stripped). The newest implementation is face-primary: every rule fires off facial expression + behavioural signals first, voice is consulted only as a high-confidence tie-breaker at the end.

```

1
2 def infer_state(self, expression: str, response_time: float,
3                 correct: bool, answer_text: str,
4                 vocal_emotion: str = "neutral",
5                 vocal_conf: float = 0.0,
6                 volume_rms: float = 0.0) -> InferredState:
7     """
8     Infer the user's state from all signals.
9     Face is primary; voice is only derived when face is Neutral and
10    the voice signal is high-confidence and not "fearful" (the MLP
11    collapses to "fearful" in silence).
12    """
13    correctness = self.rolling_correctness()
14    clean = answer_text.strip().lower()
15    is_silent = (not clean or clean in {"i don't know", "skip", "pass", "next"}) # if no
        ↳ meaningful input || input matches a skip-command phrase in the set (set over list
        ↳ because it's faster) then indeed silent (True)
16
17    # Arousal bounds calibrated against ambient noise

```

```

18     high_arousal = volume_rms > self.VOLUME_LOUD
19     low_arousal = 0 < volume_rms < self.VOLUME_QUIET
20
21     # voice trusted only when not-fearful
22     trust_voice = (vocal_conf >= 0.9 and vocal_emotion != "fearful")
23
24     if is_silent:
25         self.consecutive_silences += 1
26     else:
27         self.consecutive_silences = 0
28     if correct:
29         self.consecutive_correct += 1
30         self.consecutive_wrong = 0
31     else:
32         self.consecutive_wrong += 1
33         self.consecutive_correct = 0
34
35     # 1- FACE-PRIMARY RULES: these fire before voice is ever consulted
36
37     # thriving: good performance + fast responses
38     if (correctness >= CORRECTNESS_CEILING and response_time < RESPONSE_TIME_BASELINE * 0.5):
39         return InferredState.THRIVING
40     if expression == "Angry" and correct and response_time < RESPONSE_TIME_BASELINE * 0.6:
41         return InferredState.COMFORTABLE
42
43     # disengaged: silence + slow + poor performance
44     if self.consecutive_silences >= SILENCE_THRESHOLD:
45         return InferredState.DISENGAGED
46     if (expression == "Neutral"
47         and response_time > RESPONSE_TIME_BASELINE
48         and correctness < 0.5):
49         return InferredState.DISENGAGED
50     if (low_arousal and expression == "Neutral"
51         and response_time > RESPONSE_TIME_BASELINE * 0.8):
52         return InferredState.DISENGAGED
53
54     # frustrated: negative face + poor performance
55     if expression in ("Angry", "Disgust") and correctness < CORRECTNESS_FLOOR:
56         return InferredState.FRUSTRATED
57     if self.consecutive_wrong >= 3 and expression in ("Angry", "Sad", "Fear"):
58         return InferredState.FRUSTRATED
59     if (high_arousal and expression in ("Angry", "Disgust", "Fear")
60         and correctness < CORRECTNESS_FLOOR):
61         return InferredState.FRUSTRATED
62
63     # struggling: sadness + slow; poor correctness; fear + wrong
64     if expression == "Sad" and response_time > RESPONSE_TIME_BASELINE * 0.7:
65         return InferredState.STRUGGLING
66     if correctness < CORRECTNESS_FLOOR:
67         return InferredState.STRUGGLING
68     if expression == "Fear" and not correct:
69         return InferredState.STRUGGLING
70
71     # 2- voice tie-breakers; only when face neutral
72     if expression == "Neutral" and trust_voice:
73         if vocal_emotion == "happy" and correct and correctness >= CORRECTNESS_CEILING:
74             return InferredState.THRIVING
75         if vocal_emotion == "calm" and correctness >= 0.5:
76             return InferredState.COMFORTABLE

```

77
78

```
return InferredState.COMFORTABLE # default: face gave no negative signal, performance is
    ↪ holding
```

The `decide()` function maps the inferred state to concrete adaptive actions, paralleling Nikolaidis, Hsu and Srinivasa’s (2017, p. 625) mutual-adaptation paradigm; Table~1 summarises the mapping.

Inferred State	Difficulty	Game Switch	Hint	Tone	LED Colour
Thriving	↑ ramp up	No	No	Energetic	Green
Comfortable	↑ if >70%	No	No	Neutral	Cyan
Struggling	↓ ease off	No	Yes	Encouraging	Yellow
Frustrated	↓↓ Easy	After 3 wrong	No	Calm	Red
Disengaged	—	After 3 checkouts	No	Energetic	Magenta

Table 1: State-to-action mapping. The adaptive engine translates each inferred state into a specific combination of difficulty adjustment, game-type switch, hint provision, tone selection, and LED colour. Arrows indicate direction of difficulty change relative to the current level, wherein the mapping applies solely to `decide()`’s engine output (the spoken-checkout path, e.g. repeated *skip*); the main-loop tiered intervention on pure-silence turns, tier-1 check-in at 3 silences and tier-2 switch at 4, bypasses `decide()` entirely.

Disengagement is flagged by three OR’d rules wherein any one trips it: 1) two consecutive silences (mic-empty or “skip”/“pass” checkout phrases), 2) neutral-face + response > 15s + rolling-correctness < 50% (the cognitive-checkout pattern), 3) low-volume + neutral-face + slow-ish response (early-warning drift, catches it before accuracy tanks). GAZE then tiers its intervention, gentle check-in at three silences, game-switch at four, wherein each tier fires at most once per silent spell.

Complementing state inference, `recommend_think_budget()` sets per-turn timing thresholds (*no-speech max*, *silence tolerance*, *recording ceiling*) by combining five signals: inferred state, facial expression, prior response-time, `consecutive_silences`, and the LLM’s `waiting` flag. Crucially, `waiting` is treated as one signal among many rather than the sole path; `request_more_time` extends the budget *additively* (+3s no-speech, +1s silence, +5s record) whilst the four behavioural signals extend independently, addressing the brittleness Desai et al. (2013) observe in single-signal systems. Round 1’s generous default (7s/2.5s/15s) is target-user-aware (aphasia, arithmetic-pause tolerance for stroke-recovery users); a defensive 20s ceiling caps `record_max_secs` under `CMD_TIMEOUT` against signal-compound edge cases.

2.3.4 Generate Layer: Dynamic Prompt Construction

Rather than constructing a fixed game prompt each round, GAZE utilises OpenAI function calling to let GPT-5.4 decide actions: `build_signal_context()` packages live signals into a context block sent with four callable tools. The LLM decides which to invoke; during natural conversation it calls none, whilst during gameplay it sequences them. Game-question generation runs as a separate JSON-only call with a variety seed plus a rolling 30-question do-not-repeat memory, hence preventing mode-collapsed targets. Answer verification is delegated to a deterministic GPT-4.1 verifier (temperature 0.0), reducing the hallucination risk Ji et al. (2023, p. 3) identify; a 10-second timeout wraps every API call in case the network stalls so Pepper falls back gracefully.

A second hallucination safeguard inside `build_signal_context()` *semantically buckets* signals (e.g. `System` ↪ `pacing: relaxed and patient` rather than raw `Think budget: 18s`), preventing the dialogue temperature (0.8) from echoing integers verbatim; a failure mode Ji et al. (2023, p. 3) classify under NLG hallucination. The answer-checking path eliminates hallucination via determinism, whilst the dialogue path eliminates it via signal abstraction.

2.3.5 Output Layer: Aligned Multimodal Response

Speech and LED state fire concurrently via threading; a celebratory gesture is additionally triggered on correct game answers. LED colours reflect the inferred state, providing a non-verbal feedback channel.

2.3.6 Adaptation Self-Evaluation and Session Persistence

GAZE exposes `evaluate_adaptation()` as the `evaluate_last_adaptation` tool, hence the LLM compares consecutive rounds to judge whether a previous adaptation helped. Session progress saves to `gaze_save.json` every round, with a belt-and-braces auto-save every two turns regardless, in case of interruption.

2.4. Outcome & System Analysis (30%; Salman)

2.5. Conclusion (10%; Alfie)

GAZE implements multi-signal emotional inference across two independent modalities (*facial expression via WS-10 CNN and vocal emotion via WS-08 MLP*) alongside speech volume/RMS, response time, answer correctness, answer text, and derived temporal signals, hence yielding a more robust user-state estimate than any single channel. The implementation hardens further by recording all four Pepper microphones, canonicalising audio to mono 16-kHz by loudest-channel selection, applying Silero-VAD to both modes, and streaming Pepper’s camera via persistent `ALVideoDevice` rather than per-turn SFTP. The hybrid architecture (GPT-5.4 dialogue paired with symbolic rule-based inference and deterministic GPT-4.1 answer verification) and adaptive game-switching directly target the novelty-decay problem Smedegaard (2019, p. 414, Experiential novelty) identifies.

The conversational architecture, wherein the LLM decides actions via function calling, positions GAZE as a social companion rather than a rigid game host. Every turn, the AdaptiveEngine’s `decide()` resolves a tone label (encouraging, celebratory, calm, energetic, neutral) from the inferred state; the LLM’s dialogue register thereby modulates live rather than holding a fixed persona, extending Tapus, Tapus and Mataric’s (2008) personality-matching paradigm from static trait-fit to dynamic state-driven adaptation.

The multi-signal approach could transfer to stroke rehabilitation re-engagement (Mataric et al., 2007), educational tutoring, or neurodivergent support. Future work could replace hand-coded thresholds with learned parameters from longitudinal data, and fine-tune both models on in-session Pepper captures. GAZE sits within Section 1’s cognitive trajectory rather than the reactive social layer alone.

2.6 Task-4 References (5%)

- Ahn, M., Brohan, A., Brown, N., et al. (2022) ‘Do As I Can, Not As I Say: Grounding Language in Robotic Affordances’, *arXiv preprint arXiv:2204.01691*. Available at: <https://arxiv.org/abs/2204.01691> (Accessed: 24 March 2026).
- Calvo, R.A. and D’Mello, S. (2010) ‘Affect detection: An interdisciplinary review of models, methods, and their applications’, *IEEE Transactions on Affective Computing*, 1(1), pp. 18–37. Available at: <https://doi.org/10.1109/t-affc.2010.1> (Accessed: 14 April 2026).
- Desai, M., Kaniarasu, P., Medvedev, M., Steinfeld, A. and Yanco, H. (2013) ‘Impact of robot failures and feedback on real-time trust’, *Journal of Human-Robot Interaction*, 2(1), pp. 251–275. Available at: <https://ieeexplore.ieee.org/document/6483596> (Accessed: 20 March 2026).
- Ekman, P. and Friesen, W.V. (1971) ‘Constants across cultures in the face and emotion’, *Journal of Personality and Social Psychology*, 17(2), pp. 124–129. Available at: <https://doi.org/10.1037/h0030377> (Accessed: 14 April 2026).
- El Ayadi, M., Kamel, M.S. and Karray, F. (2011) ‘Survey on speech emotion recognition: Features, classification schemes, and databases’, *Pattern Recognition*, 44(3), pp. 572–587. Available at: <https://doi.org/10.1016/j.patcog.2010.09.020> (Accessed: 14 April 2026).
- Garcez, A. d’A. and Lamb, L. C. (2023) ‘Neurosymbolic AI: The 3rd Wave’, *Artificial Intelligence Review*, 56, pp. 12387–12406. Available at: <https://link.springer.com/article/10.1007/s10462-023-10448-w> (Accessed: 20 March 2026).

- Ji, Z., Lee, N., Frieske, R., et al. (2023) ‘Survey of Hallucination in Natural Language Generation’, *ACM Computing Surveys*, 55(12), pp. 1–38. Available at: <https://dl.acm.org/doi/10.1145/3571730> (Accessed: 22 March 2026).
- Kahn, P.H., Jr., Freier, N.G., Kanda, T., Ishiguro, H., Ruckert, J.H., Severson, R.L. and Gill, B.T. (2008) ‘Design Patterns for Sociality in Human-Robot Interaction’, in *Proceedings of the 3rd ACM/IEEE International Conference on Human-Robot Interaction (HRI ’08)*, pp. 97–104. Available at: <https://doi.org/10.1145/1349822.1349836> (Accessed: 14 April 2026).
- Matarić, M. J., Eriksson, J., Feil-Seifer, D. J. and Winstein, C. J. (2007) ‘Socially assistive robotics for post-stroke rehabilitation’, *Journal of NeuroEngineering and Rehabilitation*, 4(5), pp. 1–9. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC1821334/> (Accessed: 25 March 2026).
- Li, S. and Deng, W. (2020) ‘Deep Facial Expression Recognition: A Survey’, *IEEE Transactions on Affective Computing*, 13(3), pp. 1195–1215. Available at: <https://doi.org/10.1109/TAFFC.2020.2981446> (Accessed: 14 April 2026).
- Nikolaidis, S., Hsu, D. and Srinivasa, S. (2017) ‘Human-robot mutual adaptation in collaborative tasks: Models and experiments’, *The International Journal of Robotics Research*, 36(5–7), pp. 618–634. Available at: <https://journals.sagepub.com/doi/10.1177/0278364917690593> (Accessed: 20 March 2026).
- Picard, R.W. (1997) ‘Affective Computing’, *MIT Media Laboratory Perceptual Computing Section Technical Report No. 321*. Available at: <https://affect.media.mit.edu/pdfs/95.picard.pdf> (Accessed: 14 April 2026).
- Poria, S., Cambria, E., Bajpai, R. and Hussain, A. (2017) ‘A review of affective computing: From unimodal analysis to multimodal fusion’, *Information Fusion*, 37, pp. 98–125. Available at: <https://doi.org/10.1016/j.inffus.2017.02.003> (Accessed: 14 April 2026).
- Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2023) ‘Robust Speech Recognition via Large-Scale Weak Supervision’, in *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, PMLR 202, pp. 28492–28518. Available at: <https://proceedings.mlr.press/v202/radford23a/radford23a.pdf> *Proceedings of Machine Learning Research* (Accessed: 14 April 2026).
- Smedegaard, C. V. (2019) ‘Reframing the Role of Novelty within Social HRI: From Noise to Information’, in *Proceedings of the 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI ’19)*, pp. 411–420. Available at: <https://dl.acm.org/doi/10.1109/HRI.2019.8673219> (Accessed: 22 March 2026).
- Spezialetti, M., Placidi, G. and Rossi, S. (2020) ‘Emotion Recognition for Human-Robot Interaction: Recent Advances and Future Perspectives’, *Frontiers in Robotics and AI*, 7, Article 532279. Available at: https://www.researchgate.net/publication/347520928_Emotion_Recognition_for_Human-Robot_Interaction_Recent_Advances_and_Future_Perspectives (Accessed: 14 April 2026).
- Tapus, A., Matarić, M. J. and Scassellati, B. (2007) ‘Socially assistive robotics [Grand Challenges of Robotics]’, *IEEE Robotics & Automation Magazine*, 14(1), pp. 35–42. Available at: <https://sczlab.yale.edu/sites/default/files/files/Tapus-RAM2007.pdf> (Accessed: 25 March 2026).
- Tapus, A., Țăpuș, C. and Matarić, M. J. (2008) ‘User-robot personality matching and assistive robot behavior adaptation for post-stroke rehabilitation therapy’, *Intelligent Service Robotics*, 1(2), pp. 169–183. Available at: <https://hal.science/hal-00770108/document> (Accessed: 26 March 2026).

3. Appendix

3.1 Video Demo

- YouTube link: [test](#)

3.2 AI Declaration

Solo Work	S1 - Generative AI tools have not been used for this assessment.
Assisted Work	A1 – Idea Generation and Problem Exploration Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions). The interpretation and integration of research into the assignment remain the student's responsibility.
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.
	A11 - Other uses not listed above Please specify:
Partnered Work	P1 - Generative AI tool usage has been used integrally for this assessment Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:

Figure 6: Student Declaration of AI Tool use in this Assessment Table

I declare that I've used the AI tools listed below whilst preparing this assessment. I've read and understood the University of Plymouth's policy on the use of AI tools in assessment and confirm that my use falls within

the coursework's allowed categories, i.e. **A2 (Planning and Structuring Projects)** and **A4 (Research Assistance)**.

AI Tool Used	Purpose of Use	Extent of Use
ChatGPT	Brainstorming project ideas and structuring the report (A2)	Initial brainstorming and final outline stages for GAZE
ChatGPT	Reviewing structural alignment against grading criteria and mapping word-count budgets (A2)	After and midway through section-drafting
ChatGPT	Finding relevant pages to read in the paper (A4)	Few times if the paper is too long
ChatGPT	General conversations via web-search AI about prevalent papers to read about how the topics relates to others' studies (A4)	Few times at the end

- ☒ I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.
- ☒ I confirm that all details provided above are an accurate description of how AI was used for this assessment.

3.3 Novel Python Project: gaze22.1.py

```

1  """
2  GAZE: Game-Adaptive Zone of Engagement.
3
4  A Pepper (NAO) countdown-game host. The robot adapts difficulty,
5  pacing and feedback per turn from five signals (each paired with the function wherein it is
   ↳ produced):
6      1- facial expression: (CNN, WS-10)    primary    --->    capture_and_classify() /
   ↳ FacialExpressionModel.predict()
7      2- answer correctness: (performance) primary    --->    check_answer() (GPT-4.1 verifier)
8      3- response time:    (behavioural)    primary    --->    time.time() - question_start (main
   ↳ loop)
9      4- speech volume:    (RMS)            secondary  --->    local_record() / nao_record() (RMS
   ↳ per audio chunk)
10     5- vocal emotion:    (MLP, WS-08)      tie-breaker; MLP collapses to "fearful" on quiet/noisy
   ↳ audio ---> classify_speech_emotion() / SpeechEmotionModel.predict()
11     All five fan in to: AdaptiveEngine.infer_state() -> AdaptiveEngine.decide()
12
13  NOVELTY:
14  Multi-signal integration: single signals are not trusted alone. Voice is only consulted when the
15  face is Neutral, vocal-emotion confidence is >= 0.9 and the label is not "fearful"; the MLP might
16  collapse to it on quiet/noisy audio.
17
18  ARCHITECTURE: gpt-5.4 converse with access to four function-calling tools:
19      - generate_game_question (GPT-5.4 generates; validate_numbers_round stubbed return True)
20      - check_game_answer (GPT-4.1 yes/no verifier)
21      - evaluate_last_adaptation (self-evaluates previous round's strategy)
22      - request_more_time (acknowledges user's request for more think-time)
23
24  Initially used a wake-word defence but became unnecessary
25
26  `AdaptiveEngine`: five signals and adapt (change based on inference) difficulty,
   ↳ pacing/think-budget, and game-switching:
27      - GPT-4.1 for yes/no checks (check_answer, is_goodbye, resume-intent)
28      - GPT-5.4 for the main converse loop and question generation.
29
30  VARIOUS MODES:

```

```

31 1- LOCAL_MODE=true:
32   - webcam; works with Mac and Windows (no NAO connection)
33   - Mac/Windows mic
34   - local TTS; just run entirely without robot-connection because all input/output is local
    ↪ to tester's computer
35 2- LOCAL_MODE=false:
36   - Pepper over SSH;
37   - output (TTS, LEDs, gestures) on robot.
38
39 NAO-ran code must be non-indented to avoid processing issues with Pepper (Pepper's Python 2.7
    ↪ interpreter chokes on leading whitespace)
40
41 Now, everything is ran hybridly: dashboard stats, and the camera feed is streamed over TCP to
    ↪ the computer for
42 display and facial-expression inference. Pepper's camera is too slow for a responsive
    ↪ experience, and local webcam is much smoother. The microphone path is governed by
    ↪ LOCAL_MODE, but can be forced to the local machine with HYBRID_LOCAL_INPUT so the
    ↪ operator can speak from the computer whilst Pepper still does the talking.
43
44 3- HYBRID_LOCAL_INPUT (default = True):
45   - mic input is forced to the Mac despite LOCAL_MODE's preference
46   - essentially the brain works locally to offload inference and display whilst Pepper
    ↪ handles output (TTS, LEDs, gestures)
47
48 TESTING OBSERVATIONS:
49   - Pepper's onboard audio is *noisy*; Whisper (trained on podcasts, YouTube) is
    ↪ hallucination-prone on such input and is therefore gated by defences as follows:
50   - Silero VAD
51   - no_speech_prob, and a hallucination blacklist.
52
53   - Vosk wake-word ensure (has_wake_word()) was indeed wired early-in to
54   combat Whisper hallucinations on noisy Pepper audio. The
55   hybrid-required switch fixed this; local Mac audio sometimes
56   produces these, whereas Pepper's stream more-often does, hence
57   wake-word is now not necessary (transcribe(bypass_wake_word=True)).
58
59   - Pepper camera streaming (~10 fps) < local webcam
60   (~30 fps); the code therefore runs hybridly: length-prefixed RGB
61   over TCP port 9558 via pepper_video_loop/pepper_camera_receive_loop;
62   detect_thread_loop runs ~6-7 Hz; display repaints ~30 fps from
63   _preview_frame+_last_bbox.
64   - The emotion classifiers are lightweight, not clinical instruments and thus
65   robustness felt unfeasible
66
67
68 PER-PROPOSAL AUTHORSHIP:
69   Alfie: architecture, OpenAI integration, AdaptiveEngine. facial-expression pipeline,
70   Salman - game flow, gestures, LEDs, TTS pacing, save/load, testing.
71
72
73 LIVE PRE-DEMO CHECKLIST:
74 - [X] fix dashboard as it is just black when NAO
75 - [X] [X] eye colours should change
76 - [X] disregard all hallucinations until sufficient listened sentence
77 - [X] fix year (escape character hallucinations)
78 - [X] offload simpler tasks? to either computation or mini model
79 - [X] ensure facial expression inference 'logics commented sufficiently
80 - [X] ensure it notices and mitigates when user's disengaged
81 - [X] fix years being hallucinated

```

```

82 - [X] fix continuation from previous game 5 games to 3 make it says results every 3 rounds for
    ↪ example 2 out of 3 is correct and then adapts based in that
83
84 - [X] pirotise face over voice in mulit-singal inference
85 - [X] ensure it saves all the time not just at least 5
86 times
87
88 - [X] refine the arm movement its too jittery
89 - [X] after escalate directly address user's disengagent
90 - [X] make it more random for more games as currently it keeps giving mainly the same answers even
    ↪ despite how hard it is
91 - [X] make voice very not important
92 - [X] make it ask for name straight away
93
94 - [X] configure audio frequency to work better maybe for the nao (this was later resolved by
    ↪ hybrid-local-input switch to use local mic
95
96 - [X] make dashboard camera FPS much more fluid because its too slow now; fix was to make it
    ↪ stream via TCP to computer
97
98 ""
99
100 import os, re, sys, json, time, types, wave, struct, socket, textwrap, tempfile, threading,
    ↪ subprocess, unicodedata
101 import tkinter as tk
102 from dataclasses import dataclass, field
103 from enum import Enum
104 from typing import Optional
105 print("[booting...] stdlib loaded", flush=True)
106
107 import numpy as np
108 import cv2
109 from PIL import Image, ImageTk
110 print("[booting...] numpy + opencv loaded", flush=True)
111
112 import paramiko
113 print("[booting...] paramiko loaded", flush=True)
114
115 import sounddevice as sd
116 import librosa
117 import soundfile as sf
118 # Vosk wake-word gate
119 try:
120     from vosk import Model as VoskModel, KaldiRecognizer
121     _vosk_import_ok = True
122 except ImportError:
123     VoskModel = None
124     KaldiRecognizer = None
125     _vosk_import_ok = False
126 # Silero VAD; pre-Whisper speech gate
127 try:
128     from silero_vad import load_silero_vad, read_audio, get_speech_timestamps
129     _silero_import_ok = True
130 except ImportError:
131     load_silero_vad = None
132     read_audio = None
133     get_speech_timestamps = None
134     _silero_import_ok = False
135 print("[booting...] audio stack loaded", flush=True)

```

```

136
137 from dotenv import load_dotenv
138 from openai import OpenAI
139 print("[booting...] openai loaded", flush=True)
140
141 import joblib
142 print("[booting...] loading tensorflow (this is slow on first run)...", flush=True)
143 import tensorflow as tf
144 from tensorflow.keras.models import model_from_json
145 print("[booting...] tensorflow loaded", flush=True)
146
147 # -----
    ↪ -----
148
149 # Silero VAD loader; optional dep
150 _silero_model = None
151 if _silero_import_ok:
152     try:
153         _silero_model = load_silero_vad()
154         print("[booting] silero-vad loaded", flush=True)
155     except Exception as _vad_err:
156         print(f"[booting] silero-vad failed to load ({_vad_err}); VAD gate disabled", flush=True)
157
158 load_dotenv()
159
160
161 # NAO CONFIG
162 NAO_IP = os.getenv("NAO_IP", "ROBOT_IP")
163 NAO_USER = "nao"
164 NAO_PASS = "nao"
165 RECORD_MAX_SECS = 8 # ceiling (don't record longer than this)
166 RECORD_MIN_SECS = 1 # minimum recording before silence-detection
167 SILENCE_POLL_SECS = 0.25 # polling interval for silence detection on Pepper
168 SILENCE_DURATION = 1.2 # seconds of silence after speech to trigger stop
169 CALIBRATION_SECS = 3 # duration of start-up ambient noise calibration
170 ENERGY_BUFFER = 80 # margin above ambient baseline to set speech threshold
171 DEFAULT_ENERGY_THRESHOLD = 700 # fallback for if calibration fails
172 REMOTE_WAV = "/var/persistent/home/nao/input.wav"
173 REMOTE_IMG = "/var/persistent/home/nao/capture.jpg"
174 LOCAL_WAV = os.path.join(tempfile.gettempdir(), "gaze_input.wav")
175 LOCAL_IMG = os.path.join(tempfile.gettempdir(), "gaze_capture.jpg")
176 VOLUME_THRESHOLD = 700 # RMS; dashboard label only ("quiet" vs "normal"). Not a transcription
    ↪ gate; see NAO_MIN_RMS_TO_TRANSCRIBE.
177 NAO_MIN_RMS_TO_TRANSCRIBE = 500 # near-empty WAV floor for the NAO-mode pre-transcribe gate;
    ↪ Silero + Whisper + blacklist do the real filtering downstream.
178 FACE_CONFIDENCE_THRESHOLD = 0.5 # below: face uncertain, treat neutral
179 VOICE_CONFIDENCE_THRESHOLD = 0.5 # threshold for vocal emotion is too uncertain; treat as neutral
180 SSH_TIMEOUT = 10
181 CMD_TIMEOUT = 60
182
183 # false when connected to pepper; true for testing when no Pepper's camera
184 USE_LOCAL_CAMERA = os.getenv("GAZE_LOCAL_CAMERA", "false").lower() == "true"
185
186 # uses local webcam; Mac microphone, and macOS TTS instead of Pepper-hardware
187 LOCAL_MODE = os.getenv("GAZE_LOCAL_MODE", "false").lower() == "true"
188 if LOCAL_MODE:
189     USE_LOCAL_CAMERA = True
190
191 # Hybrid: TTS, LEDs and gestures stay on Pepper (governed by LOCAL_MODE),

```

```

192 # but the microphone path is forced to the Mac so the operator can speak from
193 # the computer whilst Pepper still does the talking. Set HYBRID_LOCAL_INPUT
194 # False to fall back to whatever LOCAL_MODE prescribes for input
195 HYBRID_LOCAL_INPUT = True
196 INPUT_IS_LOCAL = LOCAL_MODE or HYBRID_LOCAL_INPUT
197
198 DEBUG_PREVIEW = LOCAL_MODE or HYBRID_LOCAL_INPUT
199 _last_rms = 0.0 # shared with recording thread for overlay
200 _last_emotion = "" # updated by capture_and_classify
201
202 _preview_lock = threading.Lock()
203 _preview_state = {"emotion": "Neutral", "confidence": 0.0}
204 _preview_frame = None # latest RAW BGR frame (annotation now composited in camera_refresh)
205 _last_bbox = None # latest detected face bbox (x, y, w, h) in raw-frame coords; None if no face
206 DETECT_INTERVAL_SECS = 0.15
207
208 # pre-trained model paths; checks models/ first (portable), then workshop dir (dev fallback)
209 SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
210 MODELS_DIR = os.path.join(SCRIPT_DIR, "models")
211 WORKSHOP_DIR = os.path.join(SCRIPT_DIR, "..", "..", "..", "learning", "workshops") # go backwards
212     ↪ to root of repo then go to learning/workshops
213
214 def find_model(local_name, workshop_subpath):
215     "Resolve a model local models/ dir first, then workshop fallback."
216     local = os.path.join(MODELS_DIR, local_name)
217     if os.path.exists(local):
218         return local
219     return os.path.join(WORKSHOP_DIR, workshop_subpath)
220
221 MODEL_JSON = find_model("model.json", os.path.join("[X]-facial-expression-detection",
222     ↪ "model.json"))
223
224 MODEL_WEIGHTS = find_model("model_weights.weights.h5",
225     ↪ os.path.join("[X]-facial-expression-detection", "model_weights.weights.h5"))
226
227 HAAR_CASCADE = find_model("haarcascade_frontalface_default.xml", os.path.join("[X]-ws-10",
228     ↪ "haarcascade_frontalface_default.xml"))
229
230 SPEECH_MODEL = os.path.join(SCRIPT_DIR, "speech_emotion_model.pkl")
231 VOSK_MODEL_DIR = os.path.join(MODELS_DIR, "vosk-model-small-en-us-0.15")
232
233 # Vosk wake-word; "Pepper" or "Gaze"
234 _vosk_model = None
235 if _vosk_import_ok:
236     try:
237         _vosk_model = VoskModel(VOSK_MODEL_DIR)
238         print("[booting] vosk wake-word model loaded", flush=True)
239     except Exception as _vosk_err:
240         print(f"[booting] vosk failed to load ({_vosk_err}); wake-word gate disabled", flush=True)
241
242 RESPONSE_TIME_BASELINE = 15.0 # seconds; sits below 20s recording cap
243 CORRECTNESS_WINDOW = 5 # rolling window size
244 CORRECTNESS_FLOOR = 0.4 # below: ease off
245 CORRECTNESS_CEILING = 0.8 # above: ramp up
246 SILENCE_THRESHOLD = 2 # consecutive non-responses before intervention
247
248 SAVE_FILE = os.path.join(SCRIPT_DIR, "gaze_save.json")
249
250 # ensure AI key is set
251 if not os.getenv("OPENAI_API_KEY", "").strip():
252     raise SystemExit("ERROR: OPENAI_API_KEY not set. Add it to .env")
253 client = OpenAI()

```



```

247
248
249 class FacialExpressionModel:
250     "Pre-trained CNN: 7-classed emotion classifier (48x48 greyscale input)."
```

251

```

252     EMOTIONS = ["Angry", "Disgust", "Fear", "Happy", "Neutral", "Sad", "Surprise"]
253
254     def __init__(self, model_json_path, model_weights_path):
255         with open(model_json_path, "r") as f:
256             self.model = model_from_json(f.read())
257             self.model.load_weights(model_weights_path)
258             self.model.make_predict_function()
259
260     def predict(self, img):
261         "Return (emotion_label, confidence) from the (1, 48, 48, 1) array."
262         preds = self.model.predict(img, verbose=0)
263         idx = np.argmax(preds)
264         return self.EMOTIONS[idx], float(preds[0][idx])
265
266
267 class SpeechEmotionModel:
268     "Pre-trained MLP for vocal emotion classification (WS-08)."
```

269

```

270     EMOTIONS = ["calm", "happy", "fearful", "disgust"]
271
272     def __init__(self, model_path):
273         self.model = joblib.load(model_path)
274
275     @staticmethod # static because it's also used independently in classify_speech_emotion()
276     def extract_features(wav_path: str):
277         "Extract the same MFCC/chroma/mel feature vector used in WS-08 training."
278         with sf.SoundFile(wav_path) as sound_file:
279             audio = sound_file.read(dtype="float32")
280             sample_rate = sound_file.samplerate
281
282             if audio.ndim > 1:
283                 audio = audio.mean(axis=1)
284
285             # peak-normalise; helps quieter speakers
286             audio = librosa.util.normalize(audio)
287
288             n_fft = 2048
289             if len(audio) < n_fft:
290                 return None
291
292             stft = np.abs(librosa.stft(audio, n_fft=n_fft))
293
294             mfccs = np.mean(librosa.feature.mfcc(
295                 y=audio, sr=sample_rate, n_mfcc=40).T, axis=0).flatten()
296
297             chroma = np.mean(librosa.feature.chroma_stft(
298                 S=stft, sr=sample_rate).T, axis=0).flatten()
299
300             mel = np.mean(librosa.feature.melspectrogram(
301                 y=audio, sr=sample_rate).T, axis=0).flatten()
302
303             return np.concatenate([mfccs, chroma, mel])
304
305     def predict(self, wav_path: str) -> tuple[str, float]:
```

```

306         "Return (emotion_label, confidence) from a WAV file."
307         features = self.extract_features(wav_path)
308         if features is None:
309             return "neutral", 0.0
310
311         features = features.reshape(1, -1)
312         label = self.model.predict(features)[0]
313         proba = self.model.predict_proba(features)[0]
314         confidence = float(np.max(proba))
315         return label, confidence
316
317     def classify_speech_emotion(speech_model, wav_path: str) -> tuple[str, float]:
318         "Classify the vocal emotion from a WAV file."
319         if speech_model is None:
320             return "neutral", 0.0
321         if INPUT_IS_LOCAL and not has_real_speech(wav_path):
322             return "neutral", 0.0
323         try:
324             return speech_model.predict(wav_path)
325         except Exception as e:
326             print(f"Speech emo classify failed: {e}; defaulting to neutral")
327             return "neutral", 0.0
328
329
330     class Difficulty(Enum):
331         EASY = 1
332         MEDIUM = 2
333         HARD = 3
334
335     class InferredState(Enum):
336         THRIVING = "thriving"
337         COMFORTABLE = "comfortable"
338         STRUGGLING = "struggling"
339         DISENGAGED = "disengaged" # thus re-engage
340         FRUSTRATED = "frustrated"
341
342     class GameType(Enum):
343         NUMBERS = "numbers"
344         LETTERS = "letters"
345
346     BASE_SYSTEM_PROMPT = """
347     You are GAZE: an stroke-assitive *social-companionistic* robot running on a NAO humanoid robot.
348     You are a therapeutic companion first, and an engaging game host second.
349
350     CONVERSATION GUIDELINES:
351     - Refer to yourself ONLY in the first person ('I', 'me', 'your companion').
352     - Have natural, flowing conversations with the user.
353     - You can play countdown-style games (numbers rounds and letters rounds) when the moment feels
354       ↪ right or the user asks, but do NOT force a game every single turn.
355     - Each user message includes real-time emotional signals (facial expression, vocal emotion,
356       ↪ volume, response time). Use these signals to adapt your tone and approach naturally; doN'T
357       ↪ mention the signals explicitly.
358     - Keep responses concise: 2-3 sentences maximum. Your words are spoken aloud via text-to-speech,
359       ↪ so brevity is essential.
360     - If a game is active, acknowledge the user's answer before moving on.
361     - If the user seems disengaged, try a different topic or suggest a game. (re-engage them)
362     - If the user asks for more time to think, call request_more_time and respond understandably.
363     - If the user says the game is too hard or too easy, adjust the difficulty naturally in your next
364       ↪ generate_game_question call.

```

```

360 - Be genuinely present; you are the user's social companion.
361 """
362
363 # @dataclass: typed fields auto-become __init__ params
364
365 @dataclass
366 class GameState:
367     "Essentially tracks whether a countdown game is currently active."
368     active: bool = False
369     current_question: str = ""
370     current_answer: str = ""
371     category: str = ""
372     turn_count: int = 0
373     waiting: bool = False # user asked for more time to think
374     last_answer_checked: bool = False # was a game answer checked this turn?
375     last_answer_correct: bool = False # result of the last answer check
376     # snapshot of the answered round; survives a same-chain generate_game_question overwrite
377     answered_question: str = ""
378     answered_answer: str = ""
379     answered_game_type: Optional[GameType] = None
380     answered_difficulty: Optional[Difficulty] = None
381
382 @dataclass
383 class RoundResult:
384     "Record of a single game round."
385     round_number: int
386     game_type: GameType
387     difficulty: Difficulty
388     question: str
389     user_answer: str
390     correct: bool
391     response_time: float
392     facial_expression: str
393     expression_confidence: float
394     vocal_emotion: str
395     vocal_emotion_confidence: float
396     volume_rms: float # speech loudness (arousal signal)
397     inferred_state: InferredState
398     timestamp: float = field(default_factory=time.time)
399
400 @dataclass
401 class AdaptiveDecision:
402     "Output of the adaptive engine (what to do next)"
403     difficulty: Difficulty
404     game_type: GameType
405     inferred_state: InferredState
406     switch_game: bool
407     give_hint: bool
408     give_encouragement: bool
409     tone: str # "encouraging" / "celebratory" / "calm" / "energetic" / "neutral"
410
411
412 class AdaptiveEngine:
413     "Takes all five input signals and *infers* the user's real state. The adaptive engine also
414         ↳ evaluates if its previous adaptation worked, feeding that evaluation into the next
415         ↳ round's prompt."
416
417     def __init__(self):
418         self.history: list[RoundResult] = []

```

```

417     self.current_difficulty = Difficulty.MEDIUM
418     self.current_game = GameType.NUMBERS
419     self.consecutive_silences = 0
420     self.consecutive_correct = 0
421     self.consecutive_wrong = 0
422     self.games_played: dict[GameType, int] = {g: 0 for g in GameType}
423     self.game_switch_count = 0
424     self.adaptation_log: list[dict] = []
425     self.total_correct = 0
426     self.total_rounds_played = 0 # cumulative across resumed sessions
427     self.best_streak = 0
428     # recent-questions blocklist; cap 10
429     self.recent_questions: list[str] = []
430     self.recent_answers: list[str] = [] # answer-level dedup; catches mode-collapsed targets
431     self.recent_game_types: list[str] = [] # game-type rotation hint; avoids 5-in-a-row Numbers
432     # adaptive think-budget; per-round baselines
433     self.think_budget_secs = float(RECORD_MAX_SECS) # hard ceiling
434     self.silence_tolerance_secs = float(SILENCE_DURATION) # post-speech silence
435     self.no_speech_max_secs = 5.0 # give up if no speech at all
436
437
438 @property
439 def round_number(self) -> int:
440     return len(self.history) + 1
441
442 def rolling_correctness(self) -> float:
443     recent = self.history[-CORRECTNESS_WINDOW:]
444     if not recent:
445         return 0.5 # no data -> assume middle (neutral)
446     return sum(1 for r in recent if r.correct) / len(recent)
447
448
449 VOLUME_QUIET = 200 # below this -> low arousal (quiet/disengaged)
450 VOLUME_LOUD = 2000 # above this -> high arousal (excited/frustrated)
451
452 def infer_state(self, expression: str, response_time: float,
453                 correct: bool, answer_text: str,
454                 vocal_emotion: str = "neutral",
455                 vocal_conf: float = 0.0,
456                 volume_rms: float = 0.0) -> InferredState:
457     """
458     Infer the user's state from all signals.
459     Face is primary; voice is only derived when face is Neutral and
460     the voice signal is high-confidence and not "fearful" (the MLP
461     collapses to "fearful" in silence).
462     """
463     correctness = self.rolling_correctness()
464     clean = answer_text.strip().lower()
465     is_silent = (not clean or clean in {"i don't know", "skip", "pass", "next"}) # if no
466     # meaningful input || input matches a skip-command phrase in the set (set over list
467     # because it's faster) then indeed silent (True)
468
469     # Arousal bounds calibrated against ambient noise
470     high_arousal = volume_rms > self.VOLUME_LOUD
471     low_arousal = 0 < volume_rms < self.VOLUME_QUIET
472
473     # voice trusted only when not-fearful

```

```

472     trust_voice = (vocal_conf >= 0.9 and vocal_emotion != "fearful")
473
474     if is_silent:
475         self.consecutive_silences += 1
476     else:
477         self.consecutive_silences = 0
478     if correct:
479         self.consecutive_correct += 1
480         self.consecutive_wrong = 0
481     else:
482         self.consecutive_wrong += 1
483         self.consecutive_correct = 0
484
485     # 1- FACE-PRIMARY RULES: these fire before voice is ever consulted
486
487     # thriving: good performance + fast responses
488     if (correctness >= CORRECTNESS_CEILING and response_time < RESPONSE_TIME_BASELINE * 0.5):
489         return InferredState.THRIVING
490     if expression == "Angry" and correct and response_time < RESPONSE_TIME_BASELINE * 0.6:
491         return InferredState.COMFORTABLE
492
493     # disengaged: silence + slow + poor performance
494     if self.consecutive_silences >= SILENCE_THRESHOLD:
495         return InferredState.DISENGAGED
496     if (expression == "Neutral"
497         and response_time > RESPONSE_TIME_BASELINE
498         and correctness < 0.5):
499         return InferredState.DISENGAGED
500     if (low_arousal and expression == "Neutral"
501         and response_time > RESPONSE_TIME_BASELINE * 0.8):
502         return InferredState.DISENGAGED
503
504     # frustrated: negative face + poor performance
505     if expression in ("Angry", "Disgust") and correctness < CORRECTNESS_FLOOR:
506         return InferredState.FRUSTRATED
507     if self.consecutive_wrong >= 3 and expression in ("Angry", "Sad", "Fear"):
508         return InferredState.FRUSTRATED
509     if (high_arousal and expression in ("Angry", "Disgust", "Fear")
510         and correctness < CORRECTNESS_FLOOR):
511         return InferredState.FRUSTRATED
512
513     # struggling: sadness + slow; poor correctness; fear + wrong
514     if expression == "Sad" and response_time > RESPONSE_TIME_BASELINE * 0.7:
515         return InferredState.STRUGGLING
516     if correctness < CORRECTNESS_FLOOR:
517         return InferredState.STRUGGLING
518     if expression == "Fear" and not correct:
519         return InferredState.STRUGGLING
520
521     # 2- voice tie-breakers; only when face neutral
522     if expression == "Neutral" and trust_voice:
523         if vocal_emotion == "happy" and correct and correctness >= CORRECTNESS_CEILING:
524             return InferredState.THRIVING
525         if vocal_emotion == "calm" and correctness >= 0.5:
526             return InferredState.COMFORTABLE
527
528     return InferredState.COMFORTABLE # default: face gave no negative signal, performance is
529         ↪ holding

```

```

530 # core-decision function
531 def decide(self, expression: str, expression_conf: float,
532           response_time: float, correct: bool,
533           answer_text: str,
534           vocal_emotion: str = "neutral",
535           vocal_conf: float = 0.0,
536           volume_rms: float = 0.0) -> AdaptiveDecision:
537     "Return what to do next based on the inferred state."
538     state = self.infer_state(expression, response_time, correct, answer_text,
539                             vocal_emotion, vocal_conf=vocal_conf,
540                             volume_rms=volume_rms)
541     correctness = self.rolling_correctness()
542
543     new_difficulty = self.current_difficulty
544     new_game = self.current_game
545     switch_game = False
546     give_hint = False
547     give_encouragement = False
548     tone = "neutral"
549
550     if state == InferredState.THRIVING:
551         if self.current_difficulty != Difficulty.HARD:
552             new_difficulty = Difficulty(self.current_difficulty.value + 1)
553             tone = "energetic"
554         if self.consecutive_correct >= 3:
555             give_encouragement = True # acknowledge streak
556
557     elif state == InferredState.COMFORTABLE:
558         if correctness > 0.7 and self.current_difficulty != Difficulty.HARD:
559             new_difficulty = Difficulty(self.current_difficulty.value + 1)
560             tone = "neutral"
561
562     elif state == InferredState.STRUGGLING:
563         if self.current_difficulty != Difficulty.EASY:
564             new_difficulty = Difficulty(self.current_difficulty.value - 1)
565             give_hint = True
566             give_encouragement = True
567             tone = "encouraging"
568
569     elif state == InferredState.FRUSTRATED:
570         new_difficulty = Difficulty.EASY
571         give_encouragement = True
572         tone = "calm"
573         if self.consecutive_wrong >= 3:
574             switch_game = True
575             new_game = self.pick_different_game()
576
577     elif state == InferredState.DISENGAGED:
578         tone = "energetic" # robot be engaging
579         give_encouragement = True
580         if self.consecutive_silences >= 3:
581             switch_game = True
582             new_game = self.pick_different_game()
583
584     self.current_difficulty = new_difficulty
585     if switch_game:
586         self.current_game = new_game
587         self.game_switch_count += 1
588

```

```

589     self.adaptation_log.append({
590         "round": self.round_number,
591         "state": state.value,
592         "action": {"difficulty": new_difficulty.name,
593                     "switch": switch_game,
594                     "hint": give_hint,
595                     "encouragement": give_encouragement},
596     })
597
598     return AdaptiveDecision(
599         difficulty=new_difficulty, game_type=self.current_game,
600         inferred_state=state, switch_game=switch_game,
601         give_hint=give_hint, give_encouragement=give_encouragement,
602         tone=tone,
603     )
604
605 def recommend_think_budget(self, state: InferredState, expression: str,
606                           prev_response_time: float,
607                           consecutive_silences: int, waiting: bool
608                           ) -> tuple[float, float, float]:
609     """
610     Choose how long to wait for the user this turn.
611     Slower or struggling users get more time, so a long pause does not
612     flip the system to "disengaged" too quickly.
613     Returns (no_speech_max, silence_secs, record_max_secs).
614     """
615     # baseline budget (fast-track: thriving / comfortable)
616     no_speech_max = 5.0
617     silence_secs = float(SILENCE_DURATION)
618     record_max_secs = float(RECORD_MAX_SECS)
619
620     # round 1: stroke-recovery silence tolerance
621     if not self.history:
622         no_speech_max = max(no_speech_max, 7.0)
623         silence_secs = max(silence_secs, 2.5)
624         record_max_secs = max(record_max_secs, 15.0)
625
626     if state in (InferredState.STRUGGLING, InferredState.FRUSTRATED, InferredState.DISENGAGED):
627         no_speech_max = 8.0
628         silence_secs = 2.5
629         record_max_secs = 18.0
630
631     # graduated extension before disengaged flip
632     if consecutive_silences > 0:
633         no_speech_max = max(no_speech_max, 5.0 + consecutive_silences * 1.5)
634         silence_secs = max(silence_secs, 1.5 + consecutive_silences * 0.5)
635
636     if expression in ("Sad", "Fear"):
637         no_speech_max = max(no_speech_max, 7.0)
638         silence_secs = max(silence_secs, 2.0)
639
640     if prev_response_time > RESPONSE_TIME_BASELINE:
641         no_speech_max = max(no_speech_max, 7.0)
642         silence_secs = max(silence_secs, 2.0)
643
644     # LLM-flagged waiting: *honour* the signal but additively, so other cues stay weighted
645     if waiting:
646         no_speech_max += 3.0
647         silence_secs += 1.0

```

```

648         record_max_secs += 5.0
649
650         # defensive ceiling 20s; under CMD_TIMEOUT
651         record_max_secs = min(record_max_secs, 20.0)
652
653         self.no_speech_max_secs = no_speech_max
654         self.silence_tolerance_secs = silence_secs
655         self.think_budget_secs = record_max_secs
656
657         return no_speech_max, silence_secs, record_max_secs
658
659     def record_round(self, result: RoundResult):
660         self.history.append(result)
661         self.total_rounds_played += 1
662         self.games_played[result.game_type] = (
663             self.games_played.get(result.game_type, 0) + 1
664         )
665         if result.correct:
666             self.total_correct += 1
667         self.best_streak = max(self.best_streak, self.consecutive_correct)
668
669     def pick_different_game(self) -> GameType:
670         if self.current_game == GameType.NUMBERS:
671             return GameType.LETTERS
672         return GameType.NUMBERS
673
674     def get_session_summary(self) -> dict:
675         if not self.history:
676             return {"rounds": 0}
677         total = len(self.history)
678         correct = sum(1 for r in self.history if r.correct)
679         return {
680             "rounds": total,
681             "correct": correct,
682             "accuracy": round(correct / total, 2),
683             "avg_response_time": round(sum(r.response_time for r in self.history) / total, 1),
684             "games_played": {g.value: c for g, c in self.games_played.items() if c > 0},
685             "game_switches": self.game_switch_count,
686             "best_streak": self.best_streak,
687             "final_difficulty": self.current_difficulty.name,
688         }
689
690     # self-evaluative adaptation
691
692     def evaluate_adaptation(self) -> Optional[str]:
693         "Evaluate whether the previous round's adaptation worked."
694         if len(self.adaptation_log) < 2 or len(self.history) < 2:
695             return None
696
697         prev_strategy = self.adaptation_log[-2]
698         curr_strategy = self.adaptation_log[-1]
699         prev_round = self.history[-2]
700         curr_round = self.history[-1]
701
702         prev_state = prev_strategy["state"]
703         curr_state = curr_strategy["state"]
704         prev_action = prev_strategy["action"]
705
706         evaluations = []

```



```

707
708     if prev_state in ("struggling", "frustrated"):
709         if curr_round.correct and not prev_round.correct:
710             evaluations.append("Previous adaptation WORKED: lowered difficulty and user answered
711                 ↪ correctly this round (was incorrect before).")
712         elif not curr_round.correct:
713             evaluations.append("Previous adaptation DID NOT HELP YET: user still struggling
714                 ↪ despite easier difficulty. Consider providing more support.")
715
716     # Did a difficulty increase overshoot for a thriving user?
717     if prev_state == "thriving" and prev_action["difficulty"] == "HARD":
718         if curr_round.correct:
719             evaluations.append("Previous adaptation WORKED: increased difficulty and user is
720                 ↪ still performing well.")
721         elif not curr_round.correct:
722             evaluations.append("Previous adaptation OVERSHOT: increased difficulty but user got
723                 ↪ it wrong. Might need to ease back.")
724
725     # Did a re-engagement attempt work for a disengaged user?
726     if prev_state == "disengaged":
727         if curr_state != "disengaged":
728             evaluations.append(f"Previous adaptation WORKED: user was disengaged but is now
729                 ↪ {curr_state}. Re-engagement WAS effective.")
730         else:
731             evaluations.append("Previous adaptation DID NOT HELP: user remains disengaged. Try a
732                 ↪ different approach or switch game type.")
733
734     if prev_action.get("switch"):
735         if curr_state in ("thriving", "comfortable"):
736             evaluations.append(
737                 "Game switch WORKED: user transitioned to a positive state."
738             )
739         elif curr_state in ("struggling", "frustrated", "disengaged"):
740             evaluations.append(
741                 "Game switch DID NOT HELP: user is still in a negative state."
742             )
743
744     if (prev_action.get("encouragement")
745         and prev_state in ("struggling", "frustrated")
746         and curr_round.response_time < prev_round.response_time):
747         evaluations.append(
748             "Encouragement appears effective: user responded faster this round."
749         )
750
751     if not evaluations:
752         return None
753
754     return ("Adaptation evaluation from previous round:\n"
755         + "\n".join(f"- {e}" for e in evaluations))
756
757 GAME_DESCRIPTIONS = {
758     GameType.NUMBERS: """
759     a Countdown-style numbers round. Pick SIX numbers by sampling from {1 to 100}; vary the mix
760         ↪ each round so no two rounds in a row feel identical.
761     Pick a TARGET in the range 50-999 (anywhere in that range, NOT always around 100), reachable
762         ↪ from the chosen six via +, -, *, /.
763     The user must combine the given numbers with those operators to reach the target. Each number
764         ↪ may be used at most once and it should be humanly solveable""",

```

```

757     GameType.LETTERS: ""
758     a Countdown-style letters round: give the user a set of 9 random letters (a mix of vowels and
        ↪ consonants; vary the letter set every round) and ask them to form the longest word
        ↪ possible using only those letters.
759     Each letter can only be used once"",
760 }
761
762 DIFFICULTY_DESCRIPTIONS = {
763     Difficulty.EASY: "easy (straightforward, common knowledge, single-step)",
764     Difficulty.MEDIUM: "medium (requires some thought, moderately specific)",
765     Difficulty.HARD: "hard (obscure, multi-step, requires deep knowledge)",
766 }
767
768 # SSH AND PEPPER ROBOT HELPERS: NON-INDENTED PYTHON STRINGS
769 # (adapted from lab-robot-code-fin.py i.e. when
770 # we tested OpenAI on the NAO robot perhaps too early)
771
772 def ssh_connect():
773     "Open SSH connection to Pepper and return the client."
774     ssh = paramiko.SSHClient()
775     ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
776     ssh.connect(NAO_IP, username=NAO_USER, password=NAO_PASS, timeout=SSH_TIMEOUT)
777     return ssh
778
779 def nao_run(ssh, code):
780     "Execute a Python 2 snippet on Pepper via SSH."
781     escaped = code.replace("'", "'\\'")
782     try:
783         _, stdout, _ = ssh.exec_command(f"python -c '{escaped}'", timeout=CMD_TIMEOUT)
784         return stdout.read().decode().strip()
785     except Exception as e:
786         print(f"Pepper SSH exec_command dropped: {e}")
787         return ""
788
789 #listens silent
790 #not triggered by noise
791 def nao_calibrate_ambient(ssh) -> int:
792     "Calibrate the mic energy threshold to the room."
793     try:
794         # col 0 only; Pepper rejects indented python -c
795         raw = nao_run(ssh, f"")
796 from naoqi import ALProxy
797 import time
798
799 audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
800 samples = []
801 start = time.time()
802 while (time.time() - start) < {CALIBRATION_SECS}:
803     # all four mics; side speakers else missed
804     try:
805         front = audio.getFrontMicEnergy()
806         left = audio.getLeftMicEnergy()
807         right = audio.getRightMicEnergy()
808         rear = audio.getRearMicEnergy()
809         samples.append(max(front, left, right, rear))
810     except Exception:
811         # older firmwares may expose only getFrontMicEnergy; fall back gracefully
812         samples.append(audio.getFrontMicEnergy())
813     time.sleep(0.2)

```

```

814
815 if samples:
816     avg = sum(samples) / len(samples)
817     print(int(avg))
818 else:
819     print(0)
820 """
821     ambient = int(raw) if raw.strip().isdigit() else 0
822     threshold = ambient + ENERGY_BUFFER
823     print(f" Ambient energy: {ambient}, speech threshold: {threshold}")
824     return threshold
825 except Exception as e:
826     print(f" Calibration failed ({e}), using default threshold: {DEFAULT_ENERGY_THRESHOLD}")
827     return DEFAULT_ENERGY_THRESHOLD
828
829 def nao_record(ssh, energy_threshold: int = DEFAULT_ENERGY_THRESHOLD,
830               record_max_secs: float = RECORD_MAX_SECS,
831               silence_secs: float = SILENCE_DURATION):
832     """Record audio on Pepper with dynamic silence detection.
833     - get robot's front microphone (getFrontMicEnergy) to stop recording early if silence
834       ↪ detected
835     - calibrated energy threshold to avoid false positives from ambient noise
836     - if getFrontMicEnergy is unsupported (e.g. older firmware), fall back to a safe
837       ↪ fixed-duration recording to ensure the demo still works, albeit without silence
838       ↪ detection
839     """
840     # fixed-duration fallback for old firmware
841     nao_run(ssh, f"""
842 from naoqi import ALProxy
843 import time
844
845 rec = ALProxy("ALAudioRecorder", "127.0.0.1", 9559)
846
847 rec.stopMicrophonesRecording()
848 rec.startMicrophonesRecording("{REMOTE_WAV}", "wav", 16000, [1, 1, 1, 1])
849
850 try:
851     audio = ALProxy("ALAudioDevice", "127.0.0.1", 9559)
852
853     speech_detected = False
854     silence_start = None
855     start = time.time()
856     threshold = {energy_threshold}
857
858     while True:
859         elapsed = time.time() - start
860
861         # hard ceiling; never exceed max duration
862         if elapsed >= {record_max_secs}:
863             break
864
865         # all four mics; side speakers else missed
866         try:
867             energy = max(
868                 audio.getFrontMicEnergy(),
869                 audio.getLeftMicEnergy(),
870                 audio.getRightMicEnergy(),
871                 audio.getRearMicEnergy(),
872             )

```

```

870         except Exception:
871             energy = audio.getFrontMicEnergy() # firmware fallback
872
873         if elapsed < {RECORD_MIN_SECS}:
874             # minimum recording period
875             if energy > threshold:
876                 speech_detected = True
877                 time.sleep({SILENCE_POLL_SECS})
878                 continue
879
880         if energy > threshold:
881             speech_detected = True
882             silence_start = None
883         else:
884             if speech_detected and silence_start is None:
885                 silence_start = time.time()
886             if speech_detected and silence_start is not None:
887                 if (time.time() - silence_start) >= {silence_secs}:
888                     break
889
890             time.sleep({SILENCE_POLL_SECS})
891
892     except Exception as e:
893         # firmware fallback; if getFrontMicEnergy() unsupported, fixed-duration recording keeps the
894         ↪ demo alive
895         print(" [Silence detection failed: " + str(e) + "] Falling back to fixed-duration recording")
896         time.sleep({record_max_secs})
897
898     rec.stopMicrophonesRecording()
899     """
900     sftp = ssh.open_sftp()
901     sftp.get(REMOTE_WAV, LOCAL_WAV)
902     sftp.close()
903     # no gain stage; amplified room noise
904
905     # WAV layout diagnostic; four-mic recording may land multi-channel
906     try:
907         with wave.open(LOCAL_WAV, "rb") as wf:
908             secs = wf.getnframes() / float(wf.getframerate())
909             print(f" WAV debug: channels={wf.getnchannels()}, rate={wf.getframerate()},
910                   ↪ frames={wf.getnframes()}, secs={secs:.2f}")
911     except Exception as e:
912         print(f" WAV debug failed: {e}")
913
914     # collapse to mono 16 kHz so downstream gates see a canonical layout
915     force_mono_16k_wav(LOCAL_WAV)
916
917 def force_mono_16k_wav(path: str) -> None:
918     "Convert `path` to mono 16 kHz int16 PCM."
919     try:
920         audio, sr = sf.read(path, dtype="float32")
921         if audio.size == 0:
922             return
923         if audio.ndim > 1:
924             # loudest mic; mics not phase-aligned
925             rms_by_channel = np.sqrt(np.mean(audio.astype(np.float64) ** 2, axis=0))
926             audio = audio[:, int(np.argmax(rms_by_channel))]
927         if sr != LOCAL_SAMPLE_RATE:
928             audio = librosa.resample(audio, orig_sr=sr, target_sr=LOCAL_SAMPLE_RATE)

```

```

927         audio = np.clip(audio, -1.0, 1.0)
928         sf.write(path, audio, LOCAL_SAMPLE_RATE, subtype="PCM_16")
929     except Exception as e:
930         print(f" Mono conversion skipped ({e})")
931
932     #responses arrive in one block no splitting
933     #delivered with no pauses
934     def split_into_sentences(text: str) -> list[str]:
935         "Split dialogue into sentences for speech delivery."
936         raw_segments = re.split(r'(?<=[.!?])\s+', text.strip())
937         sentences = []
938         for seg in raw_segments:
939             for line in seg.split("\n"):
940                 cleaned = line.strip()
941                 if cleaned:
942                     sentences.append(cleaned)
943         return sentences if sentences else [text.strip()]
944
945     _TTS_REPLACEMENTS = { # prevent unicode characters from breaking Pepper Text-To-Speech
946         "'": "'", "'': '",
947         "''": "'", "''': '",
948         "-": "-", "-": "-", "-": "-",
949         "...": "...", " ": " ",
950     }
951
952     def clean_for_tts(text: str) -> str:
953         "Strip gesture tags and Unicode escapes for Pepper TTS."
954         text = text or ""
955         text = re.sub(r'\[gesture:\w+\]', '', text)
956         text = unicodedata.normalize("NFKC", text)
957         for bad, good in _TTS_REPLACEMENTS.items():
958             text = text.replace(bad, good)
959         # animated-speech tags
960         text = re.sub(r'\^[A-Za-z]+(?:\[([~])*\])?', '', text)
961         # ALTextToSpeech tags
962         text = re.sub(r'\\[A-Za-z]{2,8}=[^\\]*\\', '', text)
963         # literal \uXXXX leftovers
964         text = re.sub(r'\\u[0-9a-fA-F]{4}', '', text)
965         # control characters
966         text = re.sub(r'[\x00-\x1f\x7f-\x9f]', ' ', text)
967         return re.sub(r'\s+', ' ', text).strip()
968
969     #single ssh calls
970     def nao_say(ssh, text):
971         "Speak text on Pepper with sentence-level pausing."
972         text = clean_for_tts(text)
973         sentences = split_into_sentences(text)
974         safe_sentences = json.dumps(sentences)
975
976         nao_run(ssh, f"""
977 from naoqi import ALProxy
978 import time
979
980 try:
981     tts = ALProxy("ALTextToSpeech", "127.0.0.1", 9559)
982     sentences = {safe_sentences}
983
984     for i, sentence in enumerate(sentences):
985         tts.say(sentence)

```

```

986         if i < len(sentences) - 1:
987             time.sleep(0.4)
988     except Exception as e:
989         print(" [TTS failed] " + str(e))
990     """)
991
992     #animations with speech
993     def nao_say_animated(ssh, text):
994         "Try animated speech; fall back to plain TTS."
995         safe = json.dumps(text)
996         try:
997             nao_run(ssh, f"""
998 from naoqi import ALProxy
999 ALProxy("ALAnimatedSpeech","127.0.0.1",9559).say({safe})
1000 """)
1001         except Exception as e:
1002             print(f" Animated speech failed mid-sentence: {e}")
1003             nao_say(ssh, text)
1004
1005     def nao_capture_image(ssh):
1006         "Capture a photo from Pepper's camera and download it. Kept as a fallback for one-off stills;
1007         ↪ the live dashboard now uses pepper_video_loop()."
1008         nao_run(ssh, f"""
1009 from naoqi import ALProxy
1010 pc = ALProxy("ALPhotoCapture","127.0.0.1",9559)
1011 pc.setResolution(2)
1012 pc.setPictureFormat("jpg")
1013 pc.takePicture("{os.path.dirname(REMOTE_IMG)}/",
1014 ↪ "{os.path.splitext(os.path.basename(REMOTE_IMG))[0]}")
1015 """)
1016         sftp = ssh.open_sftp()
1017         sftp.get(REMOTE_IMG, LOCAL_IMG)
1018         sftp.close()
1019
1020     PEPPER_VIDEO_PORT = 9558
1021     _pepper_video_channel = None
1022
1023     def pepper_video_loop(ssh):
1024         """New persistent ALVideoDevice on Pepper streaming length-prefixed RGB frames over
1025         TCP (Transmission Control Protocol)"""
1026         global _pepper_video_channel
1027
1028         code = textwrap.dedent('''
1029             from naoqi import ALProxy
1030             import socket, struct, time
1031
1032             HOST = "0.0.0.0"
1033             PORT = {port}
1034
1035             cam = ALProxy("ALVideoDevice", "127.0.0.1", 9559)
1036             # camera_id=0 (top), resolution=1 (320x240), colour_space=11 (RGB), fps=10
1037             name = cam.subscribeCamera("gaze_video", 0, 1, 11, 10)
1038
1039             server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
1040             server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
1041             server.bind((HOST, PORT))
1042             server.listen(1)
1043
1044             conn, addr = server.accept()

```

```

1043
1044     try:
1045         while True:
1046             img = cam.getImageRemote(name)
1047             if img is None:
1048                 time.sleep(0.05)
1049                 continue
1050             width = img[0]
1051             height = img[1]
1052             data = img[6]
1053             payload = struct.pack("!II", width, height) + data
1054             conn.sendall(struct.pack("!I", len(payload)) + payload)
1055             time.sleep(0.1)
1056         finally:
1057             try:
1058                 cam.unsubscribe(name)
1059             except Exception:
1060                 pass
1061             try:
1062                 conn.close()
1063             except Exception:
1064                 pass
1065             server.close()
1066     '').format(port=PEPPER_VIDEO_PORT)
1067
1068     escaped = code.replace("'", "'\\'")
1069     # async; nao_run() reads stdout, would otherwise block
1070     _stdin, stdout, _stderr = ssh.exec_command(f"python -c '{escaped}'")
1071     _pepper_video_channel = stdout.channel # keep referenced so Paramiko doesn't GC the channel
1072     ↳ out from under the remote process
1073
1074 def _recv_exact(sock, n):
1075     "Receive exactly n bytes; raise if the socket closes mid-frame."
1076     data = b""
1077     while len(data) < n:
1078         packet = sock.recv(n - len(data))
1079         if not packet:
1080             raise ConnectionError("Socket closed whilst receiving Pepper camera frame")
1081         data += packet
1082     return data
1083
1084
1085 def pepper_camera_receive_loop(pepper_ip: str):
1086     "Pull length-prefixed RGB frames off Pepper:9558 into _preview_frame for detect_thread_loop to
1087     ↳ consume; ten 1-second connect-retries because Pepper-side bind() needs a beat to land
1088     ↳ after exec_command."
1089     global _preview_frame
1090
1091     sock = None
1092     for attempt in range(10):
1093         try:
1094             sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
1095             sock.connect((pepper_ip, PEPPER_VIDEO_PORT))
1096             print(f" Pepper video socket connected on attempt {attempt + 1}.")
1097             break
1098         except (ConnectionRefusedError, OSError) as e:
1099             print(f" Pepper video connect attempt {attempt + 1} failed ({e}); retrying...")
1100     try:

```

```

1099         sock.close()
1100     except Exception:
1101         pass
1102     sock = None
1103     time.sleep(1.0)
1104 if sock is None:
1105     print(" Pepper video stream unreachable; dashboard will stay black.")
1106     return
1107
1108     try:
1109         while True: # whilst running
1110             size_raw = _recv_exact(sock, 4)
1111             size = struct.unpack("!I", size_raw)[0]
1112             payload = _recv_exact(sock, size)
1113             width, height = struct.unpack("!II", payload[:8])
1114             rgb_bytes = payload[8:]
1115             rgb = np.frombuffer(rgb_bytes, dtype=np.uint8).reshape((height, width, 3))
1116             frame = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR) # rest of pipeline is BGR (OpenCV
                  ↪ convention)
1117             with _preview_lock:
1118                 _preview_frame = frame
1119     except Exception as e:
1120         print(f" Pepper video receiver loop exited: {e}")
1121     finally:
1122         try:
1123             sock.close()
1124         except Exception:
1125             pass
1126
1127 def nao_track_face(ssh, enable=True):
1128     "Toggle face tracking on Pepper."
1129     try:
1130         if enable:
1131             nao_run(ssh, """
1132 from naoqi import ALProxy
1133 ALProxy("ALFaceDetection","127.0.0.1",9559).subscribe("gaze_face")
1134 t = ALProxy("ALTracker","127.0.0.1",9559)
1135 t.registerTarget("Face", 0.1)
1136 t.track("Face")
1137 """)
1138         else:
1139             nao_run(ssh, """
1140 from naoqi import ALProxy
1141 t = ALProxy("ALTracker","127.0.0.1",9559)
1142 t.stopTracker()
1143 t.unregisterAllTargets()
1144 try:
1145     ALProxy("ALFaceDetection","127.0.0.1",9559).unsubscribe("gaze_face")
1146 except Exception as e:
1147     print(" [Face unsubscribe failed] " + str(e))
1148 """)
1149     except Exception as e:
1150         print(f" Face tracking unavailable: {e}")
1151
1152 #leds
1153 def nao_set_leds(ssh, group, colour, duration=1.0):
1154     try:
1155         nao_run(ssh, f"""
1156 from naoqi import ALProxy

```



```

1157 ALProxy("ALLeds", "127.0.0.1", 9559).fadeRGB("{group}", {colour}, {duration})
1158 """
1159     except Exception as e:
1160         print(f" LED set ignored: {e}")
1161
1162 # LOCAL MODE HELPERS
1163
1164 LOCAL_SAMPLE_RATE = 16000 # Whisper expects 16 kHz; we resample from native rate
1165
1166 LOCAL_SILENCE_RMS = 40 # default RMS; overridden by local_calibrate_ambient()
1167 _local_speech_detected = False # set by local_record(); used as transcription gate
1168 LOCAL_SILENCE_SECS = 1.2 # seconds of post-speech silence to stop recording; 1.5 → 1.2
1169     ↪ (aphasia-safe)
1169 LOCAL_MIN_SECS = 1.0 # minimum recording before silence detection kicks in
1170 LOCAL_NO_SPEECH_MAX = 3.0 # stop if no speech detected at all after this many seconds; 5.0 → 3.0
1171     ↪ (dominant lag)
1171 LOCAL_ENERGY_BUFFER = 60 # margin above ambient baseline for speech detection; 50 → 25 to catch
1172     ↪ quiet speech
1172
1173 def local_calibrate_ambient() -> int:
1174     "Calibrate the local-testing (Mac) mic's ambient noise level; mirrors nao_calibrate_ambient()
1175     ↪ so LOCAL_MODE testing behaves like the robot."
1175     global LOCAL_SILENCE_RMS
1176     dev_info = sd.query_devices(kind="input")
1177     dev_index = dev_info["index"]
1178     native_rate = int(dev_info["default_samplerate"])
1179     chunk_size = int(native_rate * 0.2)
1180     samples = []
1181
1182     print(f"Calibrating Mac mic (stay quiet for {CALIBRATION_SECS}s)...")
1183
1184     def cb(indata, frames, time_info, status): # get stable baseline from average RMS
1185         rms = (np.mean(indata.astype(np.float64) ** 2)) ** 0.5
1186         samples.append(rms)
1187
1188     with sd.InputStream(samplerate=native_rate, channels=1, dtype="int16",
1189         blocksize=chunk_size, device=dev_index, callback=cb):
1190         time.sleep(CALIBRATION_SECS)
1191
1192     if samples:
1193         ambient = int(sum(samples) / len(samples))
1194     else:
1195         ambient = 0
1196
1197     threshold = ambient + LOCAL_ENERGY_BUFFER
1198     LOCAL_SILENCE_RMS = threshold
1199
1200     # 500 was too high for MacBook mic
1201     global VOLUME_THRESHOLD
1202     VOLUME_THRESHOLD = max(LOCAL_SILENCE_RMS * 4, 100)
1203
1204     # scale to room; static thresholds confuse loud/quiet rooms
1205     AdaptiveEngine.VOLUME_QUIET = max(ambient * 2, 200) # 200 = absolute quiet floor
1206     AdaptiveEngine.VOLUME_LOUD = max(ambient * 10, 2000)
1207
1208     print(f" Ambient RMS: {ambient}, silence threshold: {threshold}, transcription gate:
1209         ↪ {VOLUME_THRESHOLD}")
1209     print(f" Arousal thresholds: QUIET={AdaptiveEngine.VOLUME_QUIET},
1210         ↪ LOUD={AdaptiveEngine.VOLUME_LOUD}")

```

```

1210     return threshold
1211
1212 def local_record(max_secs: float = RECORD_MAX_SECS,
1213                 no_speech_max: float = LOCAL_NO_SPEECH_MAX,
1214                 silence_secs: float = LOCAL_SILENCE_SECS):
1215     "Record audio from the Mac's built-in microphone to LOCAL_WAV with silence detection mirroring
1216     ↪ Pepper's dynamic recording."
1217     # select the default input device
1218     dev_info = sd.query_devices(kind="input")
1219     dev_index = dev_info["index"]
1220     native_rate = int(dev_info["default_samplerate"])
1221     sd.default.device = (dev_index, None)
1222     chunk_size = int(native_rate * SILENCE_POLL_SECS)
1223
1224     buffer = []
1225     speech_detected = False
1226     silence_start = None
1227     elapsed = 0.0
1228
1229     print(f" Recording from Mac mic (up to {max_secs}s, stops after silence)...")
1230
1231     def callback(indata, frames, time_info, status):
1232         buffer.append(indata.copy())
1233
1234     for attempt in range(2): #one retry if PortAudio error (if the device briefly is unavailable
1235         ↪ after Pepper usage)
1236         try:
1237             with sd.InputStream(samplerate=native_rate, channels=1, dtype="int16",
1238                                blocksize=chunk_size, callback=callback):
1239                 while elapsed < max_secs:
1240                     time.sleep(SILENCE_POLL_SECS)
1241                     elapsed += SILENCE_POLL_SECS
1242
1243                     if not buffer:
1244                         continue
1245                     data = buffer[-1]
1246
1247                     rms = (np.mean(data.astype(np.float64) ** 2)) ** 0.5 # compute RMS of the latest
1248                     ↪ chunk for real-time feedback
1249                     global _last_rms
1250                     _last_rms = rms # '' (U+2593 DARK SHADE) and '' (U+2591
1251                     ↪ LIGHT SHADE) extracted from Unicode 1.1 Block Elements
1252                     print(f"\r [{elapsed:.1f}s] RMS: {rms:.0f} {' ' if rms > LOCAL_SILENCE_RMS else
1253                     ↪ ' '}", end="", flush=True)
1254
1255                     if elapsed < LOCAL_MIN_SECS:
1256                         if rms > LOCAL_SILENCE_RMS:
1257                             speech_detected = True
1258                             continue
1259
1260                     if not speech_detected and elapsed >= no_speech_max:
1261                         break
1262
1263                     if rms > LOCAL_SILENCE_RMS:
1264                         speech_detected = True
1265                         silence_start = None
1266                     else:
1267                         if speech_detected and silence_start is None:
1268                             silence_start = elapsed

```

```

1264         if speech_detected and silence_start is not None:
1265             if (elapsed - silence_start) >= silence_secs:
1266                 break
1267         break # stream ran to completion
1268     except sd.PortAudioError as e:
1269         print(f"\n [PortAudio error, attempt {attempt + 1}] {e}")
1270         if attempt == 0:
1271             time.sleep(0.5) # let CoreAudio release the device
1272             continue
1273         print(" Mic unavailable; saving silent recording and continuing.")
1274
1275     print()
1276
1277     if buffer:
1278         audio_native = np.concatenate(buffer).flatten().astype(np.float32) / 32768.0
1279         # resample to 16 kHz for Whisper
1280         audio_16k = librosa.resample(audio_native, orig_sr=native_rate, target_sr=LOCAL_SAMPLE_RATE)
1281         audio_int16 = (audio_16k * 32768.0).astype(np.int16)
1282     else:
1283         audio_int16 = np.zeros((0,), dtype=np.int16)
1284
1285     # speech-detected flag so the transcription gate can use it
1286     global _local_speech_detected
1287     _local_speech_detected = speech_detected
1288
1289     with wave.open(LOCAL_WAV, "wb") as wf:
1290         wf.setnchannels(1)
1291         wf.setsampwidth(2)
1292         wf.setframerate(LOCAL_SAMPLE_RATE)
1293         wf.writeframes(audio_int16.tobytes())
1294     print(f" Recording saved ({elapsed:.1f}s, speech={'yes' if speech_detected else 'no'})")
1295
1296 def local_say(text: str):
1297     "Speak text using host OS built-in TTS (macOS `say` or Windows SAPI)."
1298     text = clean_for_tts(text)
1299     try:
1300         if sys.platform == "win32":
1301             # Windows: SAPI via PowerShell; env-var avoids escaping
1302             subprocess.run(
1303                 ["powershell", "-NoProfile", "-Command",
1304                  "Add-Type -AssemblyName System.Speech;(New-Object"
1305                  "    ↪ System.Speech.Synthesis.SpeechSynthesizer).Speak($env:GAZE_TTS_TEXT)"],
1306                 env={**os.environ, "GAZE_TTS_TEXT": text},
1307                 check=True, timeout=30,
1308             )
1309         else:
1310             subprocess.run(["say", text], check=True, timeout=30)
1311     except Exception as e:
1312         print(f" Local TTS broke: {e}")
1313
1314 def say(ssh_tts, text):
1315     "Dispatch TTS to local or Pepper depending on mode."
1316     text = clean_for_tts(text)
1317     if LOCAL_MODE:
1318         local_say(text)
1319     else:
1320         nao_say(ssh_tts, text)
1321     time.sleep(0.5) # ensure text-to-speech (TTS) drains so it does not hear itself

```

```

1322 def record(ssh, energy_threshold,
1323             no_speech_max: float = LOCAL_NO_SPEECH_MAX,
1324             silence_secs: float = LOCAL_SILENCE_SECS,
1325             record_max_secs: float = RECORD_MAX_SECS):
1326     """Dispatch recording to local or Pepper depending on mode;
1327     per-turn think-budget set by: AdaptiveEngine.recommend_think_budget().
1328     INPUT_IS_LOCAL routes the mic to the Mac even when LOCAL_MODE is False
1329     (gaze21 hybrid: Pepper-out, Mac-in)."""
1330     if INPUT_IS_LOCAL:
1331         local_record(max_secs=record_max_secs,
1332                     no_speech_max=no_speech_max,
1333                     silence_secs=silence_secs)
1334     else:
1335         nao_record(ssh, energy_threshold,
1336                   record_max_secs=record_max_secs,
1337                   silence_secs=silence_secs)
1338
1339     # gesture mapping; motions per game/emotional context
1340     # tags: celebrate, encourage, think, wave, calm, energetic, neutral
1341
1342     GESTURE_CODE = {
1343         "wave": """
1344 from naoqi import ALProxy
1345 import time
1346 m = ALProxy("ALMotion","127.0.0.1",9559)
1347 m.setStiffnesses("RArm", 1.0)
1348 safe = ["RShoulderPitch","RShoulderRoll","RElbowYaw","RElbowRoll","RWristYaw","RHand"]
1349 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1350 time.sleep(0.2)
1351 m.angleInterpolation(["RShoulderRoll"], [-0.45], [1.5], True)
1352 time.sleep(0.2)
1353 m.angleInterpolation(safe, [-0.3, -0.3, 1.0, 1.0, 0.0, 1.0], [2.0]*6, True)
1354 for _ in range(3):
1355     m.angleInterpolation(["RWristYaw"], [0.5], [1.2], True)
1356     m.angleInterpolation(["RWristYaw"], [-0.5], [1.2], True)
1357 time.sleep(0.4)
1358 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1359 m.setStiffnesses("RArm", 0.0)
1360 """,
1361
1362         "correct_wave": """
1363 from naoqi import ALProxy
1364 import time
1365 m = ALProxy("ALMotion","127.0.0.1",9559)
1366 m.setStiffnesses("RArm", 1.0)
1367 safe = ["RShoulderPitch","RShoulderRoll","RElbowYaw","RElbowRoll","RWristYaw","RHand"]
1368 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1369 time.sleep(0.2)
1370 m.angleInterpolation(["RShoulderRoll"], [-0.45], [1.5], True)
1371 time.sleep(0.2)
1372 m.angleInterpolation(safe, [-0.3, -0.3, 1.0, 1.0, 0.0, 1.0], [2.0]*6, True)
1373 for _ in range(3):
1374     m.angleInterpolation(["RWristYaw"], [0.5], [1.2], True)
1375     m.angleInterpolation(["RWristYaw"], [-0.5], [1.2], True)
1376 time.sleep(0.4)
1377 m.angleInterpolation(safe, [1.0, -0.15, 1.2, 0.4, 0.0, 0.0], [2.0]*6, True)
1378 m.setStiffnesses("RArm", 0.0)
1379 """,
1380 }

```

```

1381
1382
1383 # LED COLOUR MAP
1384 LED_COLOURS = {
1385     InferredState.THRIVING: 0x0000FF00, # green
1386     InferredState.COMFORTABLE: 0x0000FFFF, # cyan
1387     InferredState.STRUGGLING: 0x00FFFF00, # yellow
1388     InferredState.FRUSTATED: 0x00FF0000, # red
1389     InferredState.DISENGAGED: 0x00FF00FF, # magenta
1390 }
1391
1392
1393 _STATE_COLOURS = {
1394     InferredState.THRIVING: "green",
1395     InferredState.COMFORTABLE: "blue",
1396     InferredState.STRUGGLING: "orange",
1397     InferredState.FRUSTATED: "red",
1398     InferredState.DISENGAGED: "grey",
1399 }
1400
1401 def nao_gesture(ssh, gesture_type: str):
1402     "Execute a gesture on Pepper aligned to the game context."
1403     code = GESTURE_CODE.get(gesture_type, GESTURE_CODE["wave"])
1404     try:
1405         nao_run(ssh, code)
1406     except Exception as e:
1407         print(f" Gesture {gesture_type!r} did not play: {e}")
1408
1409
1410 def measure_volume() -> float:
1411     "RMS amplitude of the WAV; soundfile-based so multi-channel files collapse to mono cleanly."
1412     try:
1413         audio, _sr = sf.read(LOCAL_WAV, dtype="float32")
1414         if audio.size == 0:
1415             return 0.0
1416         if audio.ndim > 1:
1417             audio = audio.mean(axis=1)
1418         return float(np.sqrt(np.mean((audio * 32768.0) ** 2)))
1419     except Exception as e:
1420         print(f" Volume RMS calc failed: {e}")
1421         return 0.0
1422
1423 # Whisper hallucination blacklist; pre-normalised
1424 # exact-match: short fillers that would false-positive if matched as substrings.
1425 WHISPER_HALLUCINATION_EXACT = {
1426     "you", "mm", "hmm", "uh", "um",
1427     "thank you", "thanks",
1428     "    ",
1429     "    ",
1430     "    ",
1431     "    ",
1432     "    ",
1433 }
1434
1435 # fragments: if any appears anywhere in the normalised text, treat as hallucination.
1436 WHISPER_HALLUCINATION_FRAGMENTS = (
1437     "questions or comments",
1438     "questions or other problems",
1439     "post them in the comments",

```

```

1440     "in the comments section",
1441     "in the comment section",
1442     "if you like this video",
1443     "if you liked this video",
1444     "if you enjoyed the video",
1445     "if you enjoyed this video",
1446     "please subscribe",
1447     "subscribe and like",
1448     "like and subscribe",
1449     "like it and subscribe",
1450     "subscribe to our channel",
1451     "subscribe to my channel",
1452     "dont forget to subscribe",
1453     "do not forget to subscribe",
1454     "hit the like button",
1455     "smash that like button",
1456     "click the link",
1457     "link in the description",
1458     "link in the bio",
1459     "in the description below",
1460     "thanks for watching",
1461     "thank you for watching",
1462     "thank you so much for watching",
1463     "thanks so much for watching",
1464     "see you next time",
1465     "ill see you next time",
1466     "see you in the next video",
1467     "see you in the next one",
1468     # Whisper-prompt regurgitation; default prompt mentions "companion robot"
1469     "companion robot",
1470     "chats with a companion",
1471     "answers a quiz",
1472     "with a companion robot",
1473 )
1474
1475 # back-compat alias
1476 WHISPER_HALLUCINATIONS = WHISPER_HALLUCINATION_EXACT
1477
1478 def normalise_for_blacklist(text: str) -> str:
1479     "Normalise so blacklist matches independent of Whisper output."
1480     t = unicodedata.normalize("NFKC", text).lower()
1481     kept = [ch for ch in t if ch.isalnum() or ch.isspace()]
1482     return " ".join("".join(kept).split())
1483
1484 # regex blacklist; URL outros, promo boilerplate
1485 _URL_HALLUCINATION =
1486     ↪ re.compile(r"\bhttps?://|www\.\b|w+\.(?:com|org|net|au|co\.uk|google|sites)\b", re.I)
1487 _DISCLAIMER_HALLUCINATION = re.compile(r"\b(please see|visit|subscribe|like and
1488     ↪ subscribe|disclaimer|description)\b.{0,40}\b(complete|full|link|below|above)\b", re.I)
1489
1488 def is_known_hallucination(text: str) -> bool:
1489     norm = normalise_for_blacklist(text)
1490     if norm == "" or norm in WHISPER_HALLUCINATION_EXACT:
1491         return True
1492     # any fragment present anywhere in the normalised text counts
1493     if any(frag in norm for frag in WHISPER_HALLUCINATION_FRAGMENTS):
1494         return True
1495     # structural catch: Whisper leaks YouTube-description URLs on silence-amplified input
1496     if _URL_HALLUCINATION.search(text):

```

```

1497         return True
1498     if _DISCLAIMER_HALLUCINATION.search(text):
1499         return True
1500     return False
1501
1502 def has_real_speech(wav_path: str, min_speech_ms: int = 250,
1503                    threshold: float = 0.5) -> bool:
1504     "Silero VAD pre-gate; relaxed defaults so short answers (yes/Tom/six) survive."
1505     if _silero_model is None or get_speech_timestamps is None:
1506         return True
1507     try:
1508         wav = read_audio(wav_path, sampling_rate=LOCAL_SAMPLE_RATE)
1509         segments = get_speech_timestamps( # pass in the audio and model to return a list of dicts
            ↪ containing the start and end frame indices of genuine speech
            wav, _silero_model,
            sampling_rate=LOCAL_SAMPLE_RATE,
            min_speech_duration_ms=min_speech_ms,
            threshold=threshold,
            return_seconds=False,
        )
1516         return bool(segments)
1517     except Exception as e:
1518         print(f" Silero VAD check failed ({e}); falling through to Whisper")
1519         return True
1520
1521 def has_wake_word(wav_path: str) -> bool:
1522     """
1523     Vosk wake-word gate. True if "Pepper" or "Gaze" is heard in the WAV.
1524     Returns True if the Vosk model didn't load (optional dep).
1525     """
1526     if _vosk_model is None or KaldiRecognizer is None:
1527         return True
1528     try:
1529         rec = KaldiRecognizer(_vosk_model, LOCAL_SAMPLE_RATE,
1530                               json.dumps(["pepper", "gaze", "[unk]"]))
1531         with wave.open(wav_path, "rb") as wf:
1532             while True:
1533                 data = wf.readframes(4000)
1534                 if len(data) == 0:
1535                     break
1536                 rec.AcceptWaveform(data)
1537             final = json.loads(rec.FinalResult())
1538             text = (final.get("text") or "").lower()
1539             return "pepper" in text or "gaze" in text
1540     except Exception as e:
1541         print(f" Vosk wake-word check failed ({e}); falling through to Whisper")
1542         return True
1543
1544 #open ai whisperings and incase fails and in silent
1545 def transcribe(bypass_wake_word: bool = False,
1546               whisper_prompt: str = "User answers a quiz or chats with a companion robot.",
1547               record_again=None,
1548               max_hallucination_retries=None) -> str:
1549     # Silero VAD -> wake-word -> Whisper -> hallucination blacklist
1550     attempts_remaining = max_hallucination_retries
1551     while True:
1552         if INPUT_IS_LOCAL and not _local_speech_detected:
1553             return ""
1554

```

```

1555     # Silero VAD hard gate; NAO + LOCAL
1556     if not has_real_speech(LOCAL_WAV):
1557         print(" Silero VAD found no speech; skipping Whisper.")
1558         return ""
1559
1560     # Vosk wake-word gate; bypass for name prompt
1561     if not bypass_wake_word and not has_wake_word(LOCAL_WAV):
1562         print(" No wake-word detected; skipping Whisper.")
1563         return ""
1564
1565     try:
1566         with open(LOCAL_WAV, "rb") as fh:
1567             resp = client.audio.transcriptions.create(
1568                 model="whisper-1", #
1569                 file=fh, #
1570                 response_format="verbose_json", # get self-signals for hallucination detection
1571                 temperature=0.0, # zero randomness
1572                 prompt=whisper_prompt,
1573                 timeout=API_TIMEOUT,
1574             )
1575             text = (getattr(resp, "text", "") or "").strip()
1576             print(f" Whisper raw text: {text!r}")
1577
1578             # Whisper self-signals from verbose_json
1579             segments = getattr(resp, "segments", None) or []
1580             suspected_hallucination = False
1581             if segments:
1582                 no_speech_vals = [s.no_speech_prob for s in segments
1583                                   if getattr(s, "no_speech_prob", None) is not None]
1584                 logprob_vals = [s.avg_logprob for s in segments
1585                                 if getattr(s, "avg_logprob", None) is not None]
1586                 compression_vals = [s.compression_ratio for s in segments
1587                                     if getattr(s, "compression_ratio", None) is not None]
1588                 print(f" Whisper diagnostics: no_speech={no_speech_vals},
1589                       ↪ avg_logprob={logprob_vals}, compression={compression_vals}")
1590             if no_speech_vals and max(no_speech_vals) > 0.6:
1591                 print(f" Whisper flagged silence (max no_speech_prob={max(no_speech_vals):.2f});
1592                       ↪ dropping {text!r}")
1593                 return ""
1594             if logprob_vals and min(logprob_vals) < -1.3:
1595                 print(f" Whisper low-confidence (min avg_logprob={min(logprob_vals):.2f});
1596                       ↪ dropping {text!r}")
1597                 return ""
1598             # repetition-loop hallucination; flag for retry path
1599             if compression_vals and max(compression_vals) > 2.4:
1600                 print(f" Whisper repetition loop (max
1601                       ↪ compression_ratio={max(compression_vals):.2f}); flagging {text!r} as
1602                       ↪ hallucination")
1603                 suspected_hallucination = True
1604
1605             # normalised hallucination blacklist + Whisper-self-signal hallucinations
1606             if suspected_hallucination or is_known_hallucination(text):
1607                 print(f" Filtered Whisper hallucination: {text!r}")
1608                 if record_again is not None and (attempts_remaining is None or attempts_remaining >
1609                                                  ↪ 0):
1610                     if attempts_remaining is not None: # whilst more retries remain decrement each
1611                         ↪ time-taken and re-record
1612                         attempts_remaining -= 1
1613                 print(f" Disregarding hallucination; listening again (attempts left:

```



```

1607         ↪ {attempts_remaining}).")
1608     else:
1609         print(f" Disregarding hallucination; listening again.")
1610         record_again()
1611         continue
1612     return ""
1613
1614     # Strip leading "Pepper"/"Gaze" so handlers receive just the answer; \b blocks
1615     ↪ "Pepperoni"/"Gazebo" false positives..
1616     stripped = re.sub(r'(?i)^\s*(pepper|gaze)\b[.\s]*', '', text).strip()
1617     return stripped
1618 except Exception as e:
1619     print(f" Whisper transcribe failed ({e}); returning empty")
1620     return ""
1621
1622 # facial-expression pipeline
1623
1624 def capture_thread_loop(camera):
1625     """Tight capture loop; just grab the latest frame.
1626     Decoupled from detection so the dashboard runs at native fps."""
1627     global _preview_frame
1628     while True:
1629         ret, frame = camera.read()
1630         if not ret: # if camera read fails keep old frame as preview
1631             time.sleep(0.03)
1632             continue
1633         with _preview_lock:
1634             _preview_frame = frame
1635
1636 def detect_thread_loop(face_model, face_cascade):
1637     """Run cascade + CNN on a downscaled copy of the latest frame; update cached bbox + emotion.
1638     6-7 Hz is enough for a turn-based consumer."""
1639     global _last_emotion, _last_bbox
1640     while True:
1641         time.sleep(DETECT_INTERVAL_SECS)
1642         with _preview_lock:
1643             frame = _preview_frame
1644             if frame is None:
1645                 continue
1646
1647         # 2x downscale; 4x faster cascade
1648         small = cv2.resize(frame, (0, 0), fx=0.5, fy=0.5)
1649         gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
1650         faces = face_cascade.detectMultiScale(gray, 1.3, 5)
1651
1652         if len(faces) == 0:
1653             bbox = None
1654             emotion, conf = "Neutral", 0.0
1655         else:
1656             x, y, w, h = max(faces, key=lambda f: f[2] * f[3])
1657             roi = gray[y:y+h, x:x+w]
1658             resized = cv2.resize(roi, (48, 48))
1659             inp = resized[np.newaxis, :, :, np.newaxis]
1660             emotion, conf = face_model.predict(inp)
1661             # scale 'bbox' back to raw-frame coordinates (was shrunk 2x)
1662             bbox = (x*2, y*2, w*2, h*2)
1663
1664         with _preview_lock:
1665             _preview_state["emotion"] = emotion

```

```

1664         _preview_state["confidence"] = conf
1665         _last_bbox = bbox
1666         _last_emotion = emotion
1667
1668 def start_preview_thread(camera, face_model, face_cascade):
1669     "Start both the capture and the detection daemon threads."
1670     cap_t = threading.Thread(target=capture_thread_loop,
1671                             args=(camera,),
1672                             daemon=True)
1673     det_t = threading.Thread(target=detect_thread_loop,
1674                             args=(face_model, face_cascade),
1675                             daemon=True)
1676     cap_t.start()
1677     det_t.start()
1678     return cap_t, det_t
1679
1680 def capture_and_classify(ssh, face_model, face_cascade,
1681                         local_camera=None) -> tuple[str, float]:
1682     "Return (emotion, confidence). Local-camera branch reads + classifies inline; NAO branch reads
1683     ↪ cached state set by detect_thread_loop off the live Pepper video stream."
1684     # local-camera + preview thread already running: just read the cache
1685     if DEBUG_PREVIEW and local_camera is not None:
1686         with _preview_lock:
1687             return _preview_state["emotion"], _preview_state["confidence"]
1688
1689     # NAO: read cached state from detect thread
1690     if local_camera is None:
1691         with _preview_lock:
1692             return _preview_state["emotion"], _preview_state["confidence"]
1693
1694     # local-camera per-turn classification (DEBUG_PREVIEW disabled)
1695     ret, frame = local_camera.read()
1696     if not ret:
1697         print(" [Local camera failed] cv2.VideoCapture.read() returned False")
1698         return "Neutral", 0.0
1699
1700     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
1701     faces = face_cascade.detectMultiScale(gray, 1.3, 5)
1702
1703     if len(faces) == 0:
1704         bbox = None
1705         emotion, conf = "Neutral", 0.0
1706     else:
1707         x, y, w, h = max(faces, key=lambda f: f[2] * f[3])
1708         roi = gray[y:y+h, x:x+w]
1709         resized = cv2.resize(roi, (48, 48))
1710         inp = resized[np.newaxis, :, :, np.newaxis]
1711         emotion, conf = face_model.predict(inp)
1712         bbox = (x, y, w, h)
1713
1714     global _preview_frame, _last_bbox
1715     with _preview_lock:
1716         _preview_state["emotion"] = emotion
1717         _preview_state["confidence"] = conf
1718         _last_bbox = bbox
1719         _preview_frame = frame
1720
1721     return emotion, conf

```

```

1722
1723 API_TIMEOUT = 10 # 10-second timeout; prevents Pepper-freeze if OpenAI/network stalls
1724
1725 def check_answer(user_answer: str, correct_answer: str,
1726                  question_context: str) -> bool:
1727     "Verify the user's answer via GPT."
1728     if not user_answer.strip():
1729         return False
1730
1731     try:
1732         resp = client.chat.completions.create(
1733             model="gpt-4.1",
1734             messages=[{
1735                 "role": "system",
1736                 "content": "You are an answer checker. Given a question, the correct answer, and the
                             ↳ user's spoken answer, determine if the user is correct. Be lenient with
                             ↳ pronunciation, phrasing, and partial answers that demonstrate knowledge.
                             ↳ Respond with ONLY 'correct' or 'incorrect'.",
1737             }, {
1738                 "role": "user",
1739                 "content": f""Question: {question_context}
                             Correct answer: {correct_answer}
                             User's answer: {user_answer}""",
1740             }],
1741             temperature=0.0,
1742             timeout=API_TIMEOUT,
1743         )
1744         verdict = resp.choices[0].message.content.strip().lower()
1745         return verdict == "correct" # if AI says its 'correct' return verdict=correct
1746     except Exception as e:
1747         print(f" Answer verifier API failed: {e}")
1748         fallback_answer = correct_answer.lower().strip()
1749         fallback_user = user_answer.lower().strip()
1750
1751         return fallback_user == fallback_answer
1752
1753 # save/load sessions
1754
1755 def save_session(engine: AdaptiveEngine, preferred_game: Optional[GameType] = None,
1756                  quiet: bool = False):
1757     """Save session progress so user can continue later.
1758     quiet=True suppresses the "Session saved to ..." print, used when saving
1759     after every round so the log doesn't fill up with save confirmations.
1760     """
1761     data = {
1762         "total_correct": engine.total_correct,
1763         "total_rounds_played": engine.total_rounds_played,
1764         "best_streak": engine.best_streak,
1765         "games_played": {g.value: c for g, c in engine.games_played.items()},
1766         "game_switches": engine.game_switch_count,
1767         "last_difficulty": engine.current_difficulty.value,
1768         "last_game": engine.current_game.value,
1769         "preferred_game": preferred_game.value if preferred_game else None,
1770         "rounds_played": len(engine.history),
1771         "recent_questions": engine.recent_questions[-30:],
1772         "recent_answers": engine.recent_answers[-30:],
1773         "recent_game_types": engine.recent_game_types[-30:],
1774         "round_log": [
1775             {

```

```

1778         "round": r.round_number,
1779         "game": r.game_type.value,
1780         "difficulty": r.difficulty.value,
1781         "correct": r.correct,
1782         "time": round(r.response_time, 1),
1783         "expression": r.facial_expression,
1784         "vocal": r.vocal_emotion,
1785         "state": r.inferred_state.value,
1786     }
1787     for r in engine.history
1788 ],
1789 }
1790 with open(SAVE_FILE, "w") as f:
1791     json.dump(data, f, indent=2)
1792 if not quiet:
1793     print(f" Session saved to {SAVE_FILE}")
1794
1795 def load_session() -> Optional[dict]:
1796     "Load a previously saved session, if one exists."
1797     if not os.path.exists(SAVE_FILE):
1798         return None
1799     try:
1800         with open(SAVE_FILE, "r") as f:
1801             return json.load(f)
1802     except (json.JSONDecodeError, IOError) as e:
1803         print(f" Save file corrupt, ignoring: {e}")
1804         return None
1805
1806 def restore_engine(save_data: dict) -> AdaptiveEngine:
1807     "Restore engine state from saved data."
1808     engine = AdaptiveEngine()
1809     engine.total_correct = save_data.get("total_correct", 0)
1810     # legacy fallback; older saves used rounds_played instead
1811     engine.total_rounds_played = save_data.get("total_rounds_played",
1812                                                save_data.get("rounds_played", 0))
1813     engine.best_streak = save_data.get("best_streak", 0)
1814     engine.game_switch_count = save_data.get("game_switches", 0)
1815     engine.current_difficulty = Difficulty(save_data.get("last_difficulty", 2))
1816     engine.current_game = GameType(save_data.get("last_game", "numbers"))
1817     engine.recent_questions = list(save_data.get("recent_questions", []))[-30:]
1818     engine.recent_answers = list(save_data.get("recent_answers", []))[-30:]
1819     engine.recent_game_types = list(save_data.get("recent_game_types", []))[-30:]
1820     for g_val, count in save_data.get("games_played", {}).items():
1821         try:
1822             engine.games_played[GameType(g_val)] = count
1823         except ValueError as e:
1824             print(f" Could not restore game type {g_val!r}: {e}")
1825     return engine
1826
1827 def delete_save():
1828     "Remove the save file after a completed session or on user request."
1829     if os.path.exists(SAVE_FILE):
1830         os.remove(SAVE_FILE)
1831
1832 # OpenAI function-calling + conversation helpers
1833 '''
1834 - generate_game_question: called by the LLM when it wants to ask a new question; returns a
1835     ↪ question + correct answer for the specified game type and difficulty
1836 - check_game_answer: called by the LLM after the user answers a question; returns answer

```

```

1836         ↪ correctness if so
1837     - evaluate_last_adaptation: called by the LLM to self-evaluate if the previous round's
1838       ↪ adaptive strategy did indeed help the user
1839
1840     - request_more_time: called by the LLM when the user asks for more time
1841 '''
1842 TOOLS = [
1843     {
1844         "type": "function",
1845         "function": {
1846             "name": "generate_game_question",
1847             "description": "Generate a new countdown-style game question. Call this when the
1848               ↪ conversation naturally leads to playing a game.",
1849             "parameters": {
1850                 "type": "object",
1851                 "properties": {
1852                     "game_type": {
1853                         "type": "string",
1854                         "enum": ["numbers", "letters"],
1855                         "description": "Type of countdown game round.",
1856                     },
1857                     "difficulty": {
1858                         "type": "string",
1859                         "enum": ["EASY", "MEDIUM", "HARD"],
1860                         "description": "Difficulty level for the question.",
1861                     },
1862                 },
1863                 "required": ["game_type", "difficulty"],
1864             },
1865         },
1866     },
1867     {
1868         "type": "function",
1869         "function": {
1870             "name": "check_game_answer",
1871             "description": "Verify whether the user's spoken answer to the current game question is
1872               ↪ correct. Call this after the user gives an answer to an active game question.",
1873             "parameters": {
1874                 "type": "object",
1875                 "properties": {
1876                     "user_answer": {
1877                         "type": "string",
1878                         "description": "What the user said.",
1879                     },
1880                     "correct_answer": {
1881                         "type": "string",
1882                         "description": "The known correct answer.",
1883                     },
1884                     "question_context": {
1885                         "type": "string",
1886                         "description": "The original question text for context.",
1887                     },
1888                 },
1889                 "required": ["user_answer", "correct_answer", "question_context"],
1890             },
1891         },
1892     },
1893 ]

```

```

1891         "type": "function",
1892         "function": {
1893             "name": "evaluate_last_adaptation",
1894             "description": "Self-evaluate whether the previous round's adaptive strategy actually
                               ↪ helped the user. Returns a natural-language evaluation.",
1895             "parameters": {"type": "object", "properties": {}},
1896         },
1897     },
1898     {
1899         "type": "function",
1900         "function": {
1901             "name": "request_more_time",
1902             "description": "The user has asked for more time to think about the current game
                               ↪ question. Acknowledge their request warmly.",
1903             "parameters": {"type": "object", "properties": {}},
1904         },
1905     },
1906 ]
1907
1908 def build_signal_context(engine: AdaptiveEngine,
1909                         expression: str, expr_conf: float,
1910                         vocal_emo: str, vocal_conf: float,
1911                         vol_rms: float, response_time: float) -> str:
1912     "Build a signal summary string injected alongside every user message so the LLM can adapt its
       ↪ behaviour to the user's real-time emotional state without explicit adaptive-engine
       ↪ instructions."
1913     correctness = engine.rolling_correctness()
1914     recent_faces = [r.facial_expression for r in engine.history[-3:]]
1915     recent_vocal = [r.vocal_emotion for r in engine.history[-3:]]
1916
1917     # think-budget (float) -> (str) word label for LLM
1918     if engine.think_budget_secs >= 17.0:
1919         pacing = "relaxed and patient"
1920     elif engine.think_budget_secs <= 13.0:
1921         pacing = "brisk and energetic"
1922     else:
1923         pacing = "standard"
1924
1925     lines = [
1926         "--- LIVE SIGNALS ---",
1927         f"Turn: {engine.round_number}",
1928         f"Face: {expression} ({expr_conf:.0%}) [PRIMARY signal: trust this first]",
1929         f"Voice: {vocal_emo} ({vocal_conf:.0%}) [advisory only; the vocal model is noisy and often
                               ↪ stuck on 'fearful']",
1930         f"Volume: {vol_rms:.0f} RMS",
1931         f"Response time: {response_time:.1f}s",
1932         f"Rolling accuracy (last {CORRECTNESS_WINDOW}): {correctness:.0%}", # overall correctness
1933         f"System pacing: {pacing}",
1934     ]
1935     if recent_faces:
1936         lines.append(f"Recent faces: {'', '.join(recent_faces)}")
1937     if recent_vocal:
1938         lines.append(f"Recent vocal: {'', '.join(recent_vocal)}")
1939
1940     return "\n".join(lines)
1941
1942 def converse(conversation: list, tools: list) -> object:
1943     "General-conversation OpenAI `gpt-5.4` chat-completion call with function calling."
1944     try:

```

```

1945     resp = client.chat.completions.create(
1946         model="gpt-5.4",
1947         messages=conversation,
1948         tools=tools,
1949         temperature=0.8, # bit of randomness/creativity for more interesting companion
1950         timeout=API_TIMEOUT,
1951     )
1952     return resp.choices[0].message
1953 except Exception as e:
1954     print(f"converse() API failed... : {e}")
1955     return types.SimpleNamespace(
1956         content="I had a brief network hiccup. Let's keep going! [gesture:think]",
1957         tool_calls=None, role="assistant")
1958
1959 def execute_tool_call(tool_name: str, tool_args: dict,
1960                       engine: AdaptiveEngine, game_state: GameState,
1961                       conversation: list,
1962                       preferred_game: Optional[GameType],
1963                       dashboard=None) -> str:
1964     "Dispatch a function-calling tool invocation and return a JSON string result."
1965     if tool_name == "generate_game_question":
1966         gt = tool_args.get("game_type", "numbers")
1967         diff = tool_args.get("difficulty", "MEDIUM")
1968         result = generate_game_question_internal(
1969             gt, diff,
1970             recent=engine.recent_questions,
1971             recent_answers=engine.recent_answers,
1972             recent_game_types=engine.recent_game_types,
1973         )
1974         # dedup: question + answer + game_type
1975         new_q = result.get("question", "").strip()
1976         new_a = str(result.get("answer", "")).strip()
1977         if new_q:
1978             engine.recent_questions.append(new_q)
1979             engine.recent_questions = engine.recent_questions[-30:]
1980         if new_a:
1981             engine.recent_answers.append(new_a)
1982             engine.recent_answers = engine.recent_answers[-30:]
1983         engine.recent_game_types.append(gt)
1984         engine.recent_game_types = engine.recent_game_types[-30:]
1985         # sync adaptive engine with LLM's chosen difficulty so the engine's next decide() starts
1986         ↪ from correct baseline
1987         try:
1988             engine.current_difficulty = Difficulty[diff]
1989         except (KeyError, ValueError):
1990             pass
1991         # also sync current_game; record_round logs game_type from this
1992         try:
1993             engine.current_game = GameType(gt)
1994         except ValueError:
1995             pass
1996         game_state.active = True
1997         game_state.current_question = result.get("question", "")
1998         game_state.current_answer = result.get("answer", "")
1999         game_state.category = result.get("category", "")
2000         return json.dumps(result)
2001     elif tool_name == "check_game_answer":
2002         ua = tool_args.get("user_answer", "")

```

```

2003         if not game_state.active:
2004             game_state.last_answer_checked = False
2005             return json.dumps({"correct": False, "error": "no active game question"})
2006         # snapshot before same-chain regen overwrites
2007         game_state.answered_question = game_state.current_question
2008         game_state.answered_answer = game_state.current_answer
2009         game_state.answered_game_type = engine.current_game
2010         game_state.answered_difficulty = engine.current_difficulty
2011         # Python state is the source of truth, not LLM-supplied args
2012         is_correct = check_answer(
2013             ua,
2014             game_state.current_answer,
2015             game_state.current_question,
2016         )
2017         # last_answer_checked: ensures record_round() runs
2018         game_state.last_answer_checked = True
2019         game_state.last_answer_correct = is_correct
2020         if is_correct:
2021             game_state.active = False
2022         return json.dumps({"correct": is_correct})
2023
2024     elif tool_name == "evaluate_last_adaptation":
2025         evaluation = engine.evaluate_adaptation()
2026         return json.dumps({"evaluation": evaluation})
2027
2028     elif tool_name == "request_more_time":
2029         game_state.waiting = True
2030         return json.dumps({"acknowledged": True})
2031
2032     else:
2033         return json.dumps({"error": f"Unknown tool: {tool_name}"})
2034
2035 def process_llm_response(message, conversation: list,
2036                         engine: AdaptiveEngine, game_state: GameState,
2037                         preferred_game: Optional[GameType],
2038                         dashboard=None) -> str:
2039     "Handle the LLM response, including any tool call chains."
2040     msg_dict = {"role": "assistant", "content": message.content or ""}
2041     if message.tool_calls: # if tool calls are required inject the function-call into convo
2042         ↪ history so LLM can decide persistence
2043         msg_dict["tool_calls"] = [
2044             {
2045                 "id": tc.id,
2046                 "type": "function",
2047                 "function": {"name": tc.function.name, "arguments": tc.function.arguments},
2048             }
2049             for tc in message.tool_calls # iterate over each tool call in the message
2050         ]
2051     conversation.append(msg_dict)
2052     # Cap recursive tool calls at 5 rounds to prevent infinite loops
2053     max_tool_rounds = 5
2054     current_msg = message
2055     used_answer_check = False # gate same-chain regen so engine.decide can adapt difficulty first
2056     for _ in range(max_tool_rounds):
2057         if not current_msg.tool_calls:
2058             break # kill loop if no more tool calls
2059
2060         # block same-batch regen; engine.decide() must update difficulty first
2061         batch_has_check = any(tc.function.name == "check_game_answer"

```



```

2061         for tc in current_msg.tool_calls)
2062     for tc in current_msg.tool_calls:
2063         fn_name = tc.function.name
2064         try:
2065             fn_args = json.loads(tc.function.arguments)
2066         except json.JSONDecodeError:
2067             fn_args = {}
2068
2069         if batch_has_check and fn_name == "generate_game_question":
2070             result_str = json.dumps({"skipped": True,
2071                                     "reason": "deferred until after adaptive decision"})
2072             print(f" Skipping {fn_name}: deferred after answer check")
2073         else:
2074             print(f" Calling tool {fn_name}({fn_args})")
2075             result_str = execute_tool_call(
2076                 fn_name, fn_args, engine, game_state,
2077                 conversation, preferred_game, dashboard
2078             )
2079             print(f" Tool returned: {result_str[:120]}")
2080         if fn_name == "check_game_answer":
2081             used_answer_check = True
2082             conversation.append({
2083                 "role": "tool",
2084                 "tool_call_id": tc.id,
2085                 "content": result_str,
2086             })
2087         # withhold generate_game_question after a check; defer to next turn so engine.decide can
2088         #   ↳ update difficulty first
2089     next_tools = TOOLS
2090     current_msg = converse(conversation, next_tools)
2091     resp_dict = {"role": "assistant", "content": current_msg.content or ""}
2092     if current_msg.tool_calls:
2093         resp_dict["tool_calls"] = [
2094             {
2095                 "id": tc.id,
2096                 "type": "function",
2097                 "function": {"name": tc.function.name, "arguments": tc.function.arguments},
2098             }
2099             for tc in current_msg.tool_calls
2100         ]
2101     conversation.append(resp_dict)
2102     return current_msg.content or ""
2103
2104 #look for gestures
2105 def extract_gesture(text: str) -> str:
2106     "Returns wave always -only gesture now used."
2107     return "wave"
2108 def validate_numbers_round(numbers: list, target: int) -> bool:
2109     """"Skipped -too slow for real-time use. GPT is instructed to verify its own answer.""
2110     return True
2111
2112
2113 def generate_game_question_internal(game_type_str: str, difficulty_str: str,
2114                                     recent: Optional[list[str]] = None,
2115                                     recent_answers: Optional[list[str]] = None,
2116                                     recent_game_types: Optional[list[str]] = None) -> dict:
2117     try:
2118         gt = GameType(game_type_str)

```

```

2119 except ValueError:
2120     gt = GameType.NUMBERS
2121 try:
2122     diff = Difficulty[difficulty_str]
2123 except (KeyError, ValueError):
2124     diff = Difficulty.MEDIUM
2125
2126 import secrets
2127 variety_seed = secrets.token_hex(3)
2128
2129 banned_questions = "\n".join(f"- {q}" for q in (recent or [])[-30:])
2130 banned_answers = ", ".join((recent_answers or [])[-30:])
2131
2132 prompt = f"""You are generating a {game_type_str.upper()} round for a Countdown-style game.
2133
2134     GAME TYPE: {game_type_str.upper()} -do not generate any other type.
2135     DIFFICULTY: {DIFFICULTY_DESCRIPTIONS[diff]}
2136     VARIETY SEED (use this to pick different numbers/letters every time): {variety_seed}
2137
2138     STRICT RULES -you MUST follow all of these:
2139     1. This MUST be a {game_type_str.upper()} round. No exceptions.
2140     2. The question MUST be completely different from every question in the BANNED LIST below.
2141     3. The correct answer MUST NOT appear in the BANNED ANSWERS list below.
2142     4. Pick a fresh number set or letter set -do not reuse combinations you have used before.
2143
2144     BANNED LIST (do not repeat or closely mimic any of these):
2145     {banned_questions if banned_questions else "none yet"}
2146
2147     BANNED ANSWERS (your answer must not be any of these):
2148     {banned_answers if banned_answers else "none yet"}
2149
2150     Respond with a JSON object only -no markdown, no code fences:
2151     {{ "question": "...", "answer": "...", "category": "..." }}
2152 """
2153
2154 try:
2155     resp = client.chat.completions.create(
2156         model="gpt-5.4",
2157         messages=[
2158             {"role": "system", "content": (
2159                 "You generate countdown-style game questions. Respond ONLY with valid JSON. "
2160                 "Follow all rules in the user prompt exactly. "
2161                 "For numbers rounds you MUST verify your arithmetic before responding. "
2162                 "The 'answer' field must contain ONLY the final solution expression -"
2163                 "for example '50 + 25 = 75'. "
2164                 "Do NOT include working out, alternative attempts, explanations, "
2165                 "or commentary in the answer field. One clean expression only."
2166             )},
2167             {"role": "user", "content": prompt},
2168         ],
2169         temperature=1.2,
2170         timeout=API_TIMEOUT,
2171     )
2172
2173     content = resp.choices[0].message.content.strip()
2174     if content.startswith("```"):
2175         content = content.split("\n", 1)[1] if "\n" in content else content[3:]
2176     if content.endswith("```"):
2177         content = content[:-3]

```

```

2178         content = content.strip()
2179     result = json.loads(content)
2180
2181     # validate numbers rounds as GPT regularly hallucinates unreachable targets
2182     if game_type_str == "numbers":
2183         try:
2184             import re
2185             q = result.get("question", "")
2186             nums = list(map(int, re.findall(r'\b\d+\b', q.split("make")[0])))
2187             target_match = re.search(r'make.*?(\d+)', q)
2188             if target_match and nums:
2189                 target = int(target_match.group(1))
2190                 if not validate_numbers_round(nums, target):
2191                     print(f" GPT produced unreachable target {target} from {nums}; using
2192                         ↪ fallback")
2193                     return {
2194                         "question": "Using 25, 50, 3, 6, 4, and 2, make the target 100.",
2195                         "answer": "25 x 4 = 100",
2196                         "category": "numbers fallback",
2197                     }
2198             except Exception as ve:
2199                 print(f" Numbers validation skipped: {ve}")
2200
2201         return result
2202     except json.JSONDecodeError:
2203         return {"question": content if 'content' in locals() else "", "answer": "", "category":
2204             ↪ "general"}
2205     except Exception as e:
2206         print(f" Game question gen failed: {e}")
2207         return {
2208             "question": "Let's try a quick one: what is 25 + 17?",
2209             "answer": "42",
2210             "category": "arithmetic fallback",
2211         }
2212
2213     _STATE_COLOURS = {
2214         InferredState.THRIVING: "green",
2215         InferredState.COMFORTABLE: "blue",
2216         InferredState.STRUGGLING: "orange",
2217         InferredState.FRUSTATED: "red",
2218         InferredState.DISENGAGED: "grey",
2219     }
2220
2221     class GazeDashboard:
2222         """Tkinter dashboard for GAZE.
2223         1- create a window with two panels: left = conversation then video, right for inferred
2224             ↪ stats/signals/state
2225         2- left panel: camera canvas above the scrollable conversation log; log is read-only default
2226             ↪ and
2227             flipped to "normal" when a new line is appended
2228         3- right panel: round/score/streak counters at top, then the transcription block (what the
2229             ↪ user said + correct/incorrect,
2230             recoloured per outcome), then the live signals (face CNN, voice MLP, volume RMS, response
2231             ↪ time, rolling accuracy,
2232             think budget), then the adaptive decision block (inferred state, difficulty, tone,
2233             ↪ adaptations, plus a grey eval-note line below)
2234         4- quit handling: the Quit button, Esc, Cmd/Ctrl-Q and the window-close (X) all converge on
2235             ↪ quit_app(), so the SSH farewell + LED-off
2236         always run no matter how the user closes the dashboard

```

```

2229 5- refresh loops: camera_refresh() composites the latest raw frame with the cached detection
    ↪ overlay at native fps; signal_refresh()
2230 repaints the StringVar-bound labels at a slower cadence; both self-reschedule via
    ↪ root.after()
2231 """
2232
2233 CAMERA_W, CAMERA_H = 400, 300
2234
2235 def __init__(self, ssh=None, ssh_tts=None):
2236     # stashed for quit_app farewell + LED-off
2237     self._ssh = ssh
2238     self._ssh_tts = ssh_tts
2239     self.root = tk.Tk()
2240     self.root.title("GAZE Dashboard")
2241     self.root.resizable(False, False)
2242
2243     # left col: video + chat; right col: stats
2244     left = tk.Frame(self.root)
2245     left.grid(row=0, column=0, padx=8, pady=8, sticky="n")
2246
2247     # camera canvas; black letterboxes empty pixels
2248     self.camera_label = tk.Label(left, bg="black",
2249                                   width=self.CAMERA_W, height=self.CAMERA_H)
2250     self.camera_label.pack()
2251
2252     tk.Label(left, text="Conversation:").pack(anchor="w", pady=(6, 0))
2253     conv_frame = tk.Frame(left)
2254     conv_frame.pack()
2255     # read-only by default; helpers flip to insert
2256     self._conv_text = tk.Text(conv_frame, font=("Courier", 10),
2257                               width=50, height=10, wrap="word",
2258                               state="disabled")
2259     # scrollbar bidirectional wiring
2260     conv_scroll = tk.Scrollbar(conv_frame, command=self._conv_text.yview)
2261     self._conv_text.configure(yscrollcommand=conv_scroll.set)
2262     self._conv_text.pack(side="left", fill="both", expand=True)
2263     conv_scroll.pack(side="right", fill="y")
2264
2265     right = tk.Frame(self.root)
2266     right.grid(row=0, column=1, padx=(0, 8), pady=8, sticky="n")
2267
2268     tk.Label(right, text="GAZE (Game-Adaptive Zone of Engagement) Dashboard",
2269              font=("TkDefaultFont", 14, "bold")).pack(pady=(0, 4))
2270
2271     self._round_var = tk.StringVar(value="Round: -")
2272     self._score_var = tk.StringVar(value="Score: 0/0")
2273     self._streak_var = tk.StringVar(value="Streak: 0")
2274     for var in (self._round_var, self._score_var, self._streak_var):
2275         tk.Label(right, textvariable=var).pack(anchor="w")
2276
2277     # user transcription only; answer on stdout
2278     tk.Label(right, text="").pack()
2279     tk.Label(right, text="Transcription:",
2280              font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
2281     self._heard_var = tk.StringVar(value="You said: -")
2282     self._result_var = tk.StringVar(value="")
2283     tk.Label(right, textvariable=self._heard_var).pack(anchor="w")
2284     # fg recolour on correctness
2285     self._result_label = tk.Label(right, textvariable=self._result_var,

```

```

2286                 font=("TkDefaultFont", 11, "bold"))
2287 self._result_label.pack(anchor="w")
2288
2289 tk.Label(right, text="").pack()
2290 tk.Label(right, text="Live Signals:",
2291         font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
2292 self._face_var = tk.StringVar(value="Face (CNN): -")
2293 self._voice_var = tk.StringVar(value="Voice (MLP): -")
2294 self._vol_var = tk.StringVar(value="Volume RMS: -")
2295 self._time_var = tk.StringVar(value="Response time: -")
2296 self._acc_var = tk.StringVar(value="Rolling acc: -")
2297 self._budget_var = tk.StringVar(value="Think budget: -")
2298 for var in (self._face_var, self._voice_var, self._vol_var,
2299           self._time_var, self._acc_var, self._budget_var):
2300     tk.Label(right, textvariable=var,
2301             font=("Courier", 10)).pack(anchor="w")
2302
2303 tk.Label(right, text="").pack()
2304 tk.Label(right, text="Adaptive Decision:",
2305         font=("TkDefaultFont", 10, "bold")).pack(anchor="w")
2306 self._state_var = tk.StringVar(value="State: -")
2307 # kept for state-coloured fg
2308 self._state_label = tk.Label(right, textvariable=self._state_var,
2309                             font=("TkDefaultFont", 11, "bold"))
2310 self._state_label.pack(anchor="w")
2311
2312 self._diff_var = tk.StringVar(value="Difficulty: -")
2313 self._tone_var = tk.StringVar(value="Tone: -")
2314 self._adapt_var = tk.StringVar(value="Adaptations: -")
2315 for var in (self._diff_var, self._tone_var, self._adapt_var):
2316     tk.Label(right, textvariable=var).pack(anchor="w")
2317
2318 # adaptation-eval note (miniscule 'grey' text)
2319 self._eval_var = tk.StringVar(value="")
2320 self._eval_label = tk.Label(right, textvariable=self._eval_var,
2321                             font=("TkDefaultFont", 9), fg="grey",
2322                             wraplength=340, justify="left")
2323 self._eval_label.pack(anchor="w")
2324
2325 tk.Button(right, text="Quit (Esc)",
2326         command=self.quit_app).pack(pady=(10, 0))
2327
2328 self.root.bind("<Escape>", lambda e: self.quit_app())
2329 # Cmd-Q is macOS-only; guard each
2330 for combo in ("<Command-q>", "<Control-q>"):
2331     try:
2332         self.root.bind(combo, lambda e: self.quit_app())
2333     except tk.TclError:
2334         pass
2335 # route window-close through quit_app
2336 self.root.protocol("WM_DELETE_WINDOW", self.quit_app)
2337
2338 self.camera_refresh()
2339 self.signal_refresh()
2340 self.root.update() # paint before mainloop()
2341
2342
2343 def camera_refresh(self):
2344     """Composite the latest raw frame with the cached detection overlay.

```

```

2345 Bbox is drawn on every fresh raw frame for a live-feed."""
2346 with _preview_lock:
2347     frame = _preview_frame
2348     bbox = _last_bbox
2349     emotion = _preview_state["emotion"]
2350     conf = _preview_state["confidence"]
2351
2352 if frame is not None:
2353     display = frame.copy()                    # don't mutate the shared raw frame; the
        ↳ bbox/text overlays are per-paint
2354     if bbox is not None:
2355         x, y, w, h = bbox
2356         cv2.rectangle(display, (x, y), (x + w, y + h), (0, 255, 0), 2)
2357         label = f"{emotion} ({conf:.0%})"
2358         cv2.putText(display, label, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
2359                     0.8, (0, 255, 0), 2)
2360         rgb = cv2.cvtColor(display, cv2.COLOR_BGR2RGB) # OpenCV is BGR, Tk/PIL expect RGB; swap
        ↳ channels here
2361         small = cv2.resize(rgb, (self.CAMERA_W, self.CAMERA_H))
2362         img = ImageTk.PhotoImage(Image.fromarray(small)) # Tk-compatible image wrapper around
        ↳ the PIL image
2363         self.camera_label.configure(image=img)
2364         self.camera_label._photo = img # prevent garbage collection -Tk doesn't keep a ref to
        ↳ PhotoImage so without an attribute hold the image gets GC'd and the canvas blanks
2365
2366     self.root.after(33, self.camera_refresh) # ~30 fps target -Tk's standard self-rescheduling
        ↳ animation pattern; after(ms, fn) queues fn onto the mainloop after ms milliseconds
2367
2368 def signal_refresh(self):
2369     with _preview_lock:
2370         emotion = _preview_state["emotion"]
2371         conf = _preview_state["confidence"]
2372     self._face_var.set(f"Face (CNN): {emotion} ({conf:.0%})")
2373     self.root.after(500, self.signal_refresh) # 2 Hz
2374
2375 # thread-safe scheduling: tkinter widgets are main-thread only (macOS); root.after(0, fn)
        ↳ queues fn onto the main loop.
2376
2377 def on_main(self, fn):
2378     "Schedule fn to run on the main (tkinter) thread."
2379     try:
2380         self.root.after(0, fn)
2381     except tk.TclError:
2382         pass
2383
2384
2385 def update_robot_speech(self, text: str):
2386     # flip read-only widget to insert, scroll, re-disable
2387     def apply():
2388         self._conv_text.configure(state="normal")
2389         self._conv_text.insert("end", f"Robot: {text}\n\n")
2390         self._conv_text.see("end")
2391         self._conv_text.configure(state="disabled")
2392     self.on_main(apply) # schedule the UI update on the main thread
2393
2394 def append_user_speech(self, text: str):
2395     def apply():
2396         self._conv_text.configure(state="normal")
2397         self._conv_text.insert("end", f"You: {text}\n")

```

```

2398         self._conv_text.see("end")
2399         self._conv_text.configure(state="disabled")
2400         self._heard_var.set(f"You said: {text}") # mirror to right panel
2401     self.on_main(apply)
2402
2403     def update_think_budget(self, secs: float):
2404         "Update the dashboard's adaptive think-budget diagnostic row."
2405         def apply():
2406             self._budget_var.set(f"Think budget: {secs:.1f}s")
2407         self.on_main(apply)
2408
2409     def update_signals(self, round_num: int, user_answer: str, correct_answer: str,
2410                       correct: bool, expression: str, expr_conf: float,
2411                       vocal_emo: str, vocal_conf: float, vol_rms: float,
2412                       response_time: float, rolling_acc: float,
2413                       total_correct: int, total_rounds: int, streak: int):
2414         def apply():
2415             self._round_var.set(f"Round: {round_num}")
2416             self._score_var.set(f"Score: {total_correct}/{total_rounds}")
2417             self._streak_var.set(f"Streak: {streak}")
2418
2419             self._heard_var.set(f"You said: {user_answer if user_answer else '(silence)'}") #
2420                 ↪ extract user's answer
2421             if not correct_answer: # baseline; non-active game state
2422                 self._result_var.set("")
2423                 self._result_label.configure(fg="grey")
2424             elif correct:
2425                 self._result_var.set("CORRECT")
2426                 self._result_label.configure(fg="green")
2427             elif not user_answer:
2428                 self._result_var.set("NO ANSWER")
2429                 self._result_label.configure(fg="grey")
2430             else:
2431                 self._result_var.set("INCORRECT")
2432                 self._result_label.configure(fg="red")
2433
2434             self._face_var.set(f"Face (CNN): {expression} ({expr_conf:.0%})")
2435             self._voice_var.set(f"Voice (MLP): {vocal_emo} ({vocal_conf:.0%})")
2436
2437             vol_tag = "(loud)" if vol_rms > 2000 else "(quiet)" if vol_rms < VOLUME_THRESHOLD else
2438                 ↪ "(normal)"
2439             bar_len = min(int(vol_rms / 250), 20)
2440             meter = "#" * bar_len + "." * (20 - bar_len)
2441             self._vol_var.set(f"Volume RMS: {vol_rms:>5.0f} [{meter}] {vol_tag}")
2442             self._time_var.set(f"Response time: {response_time:.1f}s")
2443             self._acc_var.set(f"Rolling acc: {rolling_acc:.0%}")
2444
2445         self.on_main(apply)
2446
2447     def update_decision(self, decision, adaptation_eval: str = None):
2448         state = decision.inferred_state
2449         def apply():
2450             self._state_var.set(f"State: {state.value.upper()}")
2451             self._state_label.configure(fg=_STATE_COLOURS.get(state, "grey"))
2452             self._diff_var.set(f"Difficulty: {decision.difficulty.name}")
2453             self._tone_var.set(f"Tone: {decision.tone}")
2454             flags = []
2455             if decision.give_hint: flags.append("hint")
2456             if decision.give_encouragement: flags.append("encouragement")

```

```

2455         if decision.switch_game: flags.append(f"switch -> {decision.game_type.value}")
2456         self._adapt_var.set(f"Adaptations: {'', '.join(flags) if flags else 'none'}")
2457         self._eval_var.set(adaptation_eval if adaptation_eval else "")
2458     self.on_main(apply)
2459
2460     def quit_app(self):
2461         "Speak a farewell, dim Pepper's eye-LEDs, then kill the process."
2462         print("\n [Dashboard] Quit requested.")
2463         if not LOCAL_MODE and self._ssh is not None:
2464             nao_set_leds(self._ssh, "FaceLeds", 0x00000000, 0.5)
2465             # local_say in LOCAL_MODE; SSH-TTS otherwise
2466             say(self._ssh_tts, "It was great to chat! See you next time!")
2467             self.close()
2468             os._exit(0)
2469
2470     def close(self):
2471         try:
2472             self.root.destroy()
2473         except tk.TclError:
2474             pass
2475
2476     def is_goodbye(text: str) -> bool:
2477         "LLM goodbye-intent gate; True if the user signalled they want to leave."
2478         if not text:
2479             return False
2480         try:
2481             bye = client.chat.completions.create(
2482                 model="gpt-4.1", max_completion_tokens=5, temperature=0.0,
2483                 timeout=API_TIMEOUT,
2484                 messages=[{"role": "system", "content":
2485                     "Did the user just signal they want to end the conversation (e.g. 'bye', 'goodbye',
2486                     ↪ 'I'm done', 'see you later', 'I have to go')? Reply ONLY with 'GOODBYE' or
2487                     ↪ 'CONTINUE'."},
2488                     {"role": "user", "content": text}],
2489                 ).choices[0].message.content.strip().upper()
2490         except Exception:
2491             return False
2492         return "GOODBYE" in bye
2493
2494     def conversation_loop(dashboard, face_model, face_cascade, speech_model,
2495                          local_camera, ssh, ssh_tts, energy_threshold):
2496         "Comprehensive main-conversation loop."
2497         preferred_game = None
2498         engine = AdaptiveEngine()
2499         game_state = GameState()
2500
2501         save_data = load_session()
2502         if save_data:
2503             # legacy fallback; older saves only had per-session rounds_played
2504             prev_rounds = save_data.get("total_rounds_played",
2505                                         save_data.get("rounds_played", 0))
2506             prev_correct = save_data.get("total_correct", 0)
2507
2508             welcome_back = f"Welcome back! Last time you played {prev_rounds} rounds and got
2509             ↪ {prev_correct} correct. Want to continue where you left off, or start fresh?"
2510             if not LOCAL_MODE:
2511                 nao_track_face(ssh, enable=True)
2512                 nao_set_leds(ssh, "FaceLeds", 0x0000FF00, 1.0)

```



```

2511     nao_gesture(ssh, "wave")
2512 say(ssh_tts, welcome_back)
2513 print(f"\nRobot: {welcome_back}")
2514 dashboard.update_robot_speech(welcome_back)
2515
2516 print("\nListening for continue/fresh...")
2517 if not LOCAL_MODE:
2518     nao_set_leds(ssh, "EarLeds", 0x0000FF00, 0.3)
2519 record(ssh, energy_threshold,
2520         no_speech_max=4.0, silence_secs=2.0, record_max_secs=6.0)
2521
2522 expression, expr_conf = capture_and_classify(
2523     ssh, face_model, face_cascade, local_camera)
2524 vocal_emo, vocal_conf = classify_speech_emotion(speech_model, LOCAL_WAV)
2525 vol_rms = measure_volume()
2526 dashboard.update_signals(
2527     round_num=0, user_answer="", correct_answer="", correct=False,
2528     expression=expression, expr_conf=expr_conf,
2529     vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2530     vol_rms=vol_rms, response_time=0, rolling_acc=0,
2531     total_correct=0, total_rounds=0, streak=0,
2532 )
2533
2534 resume_text = transcribe(
2535     bypass_wake_word=True,
2536     record_again=lambda: record(ssh, energy_threshold,
2537                                 no_speech_max=4.0, silence_secs=2.0, record_max_secs=6.0),
2538 )
2539 if resume_text:
2540     print(f" Heard: {resume_text}")
2541     dashboard.append_user_speech(resume_text)
2542     if is_goodbye(resume_text):
2543         dashboard.quit_app()
2544 else:
2545     print(" Heard: (silence)")
2546
2547 # LLM intent classifier; on failure fall back to a keyword check, then
2548 # default to CONTINUE so an API timeout never silently nukes the save.
2549 intent = ""
2550 if resume_text:
2551     try:
2552         intent = client.chat.completions.create(
2553             model="gpt-4.1", max_completion_tokens=5, temperature=0.0,
2554             timeout=API_TIMEOUT,
2555             messages=[{"role": "system", "content":
2556                 "The user was just asked whether they want to CONTINUE their previous session
2557                 ↳ or start FRESH. Reply ONLY with 'CONTINUE' or 'FRESH'."},
2558                 {"role": "user", "content": resume_text}],
2559             ).choices[0].message.content.strip().upper()
2560     except Exception as e:
2561         print(f" Resume intent classification failed ({e}); falling back to keyword match")
2562         low = resume_text.lower()
2563         fresh_kw = ("fresh", "start over", "restart", "new game", "start again", "begin
2564             ↳ again", "from scratch")
2565         continue_kw = ("continue", "carry on", "keep going", "resume", "where i left",
2566             ↳ "where we left", "yes", "yeah", "yep")
2567         if any(k in low for k in fresh_kw):
2568             intent = "FRESH"
2569         elif any(k in low for k in continue_kw):

```

```

2567         intent = "CONTINUE"
2568     else:
2569         # ambiguous; preserve the save rather than wipe it
2570         intent = "CONTINUE"
2571         print(f" Keyword fallback resolved intent: {intent}")
2572     if "CONTINUE" in intent:
2573         engine = restore_engine(save_data)
2574         preferred_game = engine.current_game
2575         restored_rounds = save_data.get("total_rounds_played",
2576                                       save_data.get("rounds_played", 0))
2577         restore_msg = f"Restoring your previous session: {restored_rounds} rounds on record."
2578         say(ssh_tts, restore_msg)
2579         print(f"\nRobot: {restore_msg}")
2580     else:
2581         delete_save()
2582         fresh_msg = "No worries, starting fresh! Your previous save has been cleared."
2583         say(ssh_tts, fresh_msg)
2584         print(f"\nRobot: {fresh_msg}")
2585
2586
2587 if not LOCAL_MODE:
2588     nao_track_face(ssh, enable=True)
2589     nao_set_leds(ssh, "FaceLeds", 0x0000FF00, 1.0)
2590
2591 conversation = [{"role": "system", "content": BASE_SYSTEM_PROMPT}]
2592
2593 greeting_directive = "[Give a warm, supportive greeting. Mention you can chat, play games, or
2594 ↪ just hang out. Keep it to 2 sentences.]"
2595 greeting_msg = converse(
2596     conversation + [{"role": "user", "content": greeting_directive}],
2597     TOOLS,
2598 )
2599 greeting_text = greeting_msg.content or "Hello, lovely to meet you!"
2600 greeting_gesture = extract_gesture(greeting_text)
2601 greeting_speech = re.sub(r'\[gesture:\w+\]', '', greeting_text).strip()
2602
2603 conversation.append({"role": "assistant", "content": greeting_text})
2604
2605 if not LOCAL_MODE:
2606     nao_gesture(ssh, "wave")
2607     say(ssh_tts, greeting_speech)
2608     print(f"\nRobot: {greeting_speech}")
2609     dashboard.update_robot_speech(greeting_speech)
2610
2611 turn_count = 0
2612 # tiered disengagement; check-in at 3, game-switch at 4
2613 nudge_level = 0
2614
2615 try:
2616     while True:
2617         turn_count += 1
2618         print(f"\n{'-' * 40} Turn {turn_count} {'-' * 40}")
2619
2620         print("Capturing expression...")
2621         expr_conf = capture_and_classify(
2622             ssh, face_model, face_cascade, local_camera
2623         )
2624         if expr_conf < FACE_CONFIDENCE_THRESHOLD:

```

```

2625         print(f" Expression: {expression} ({expr_conf:.2f}): LOW CONFIDENCE, treating as
           ↪ Neutral")
2626         expression = "Neutral"
2627     else:
2628         print(f" Expression: {expression} ({expr_conf:.2f})")
2629
2630     question_start = time.time()
2631     print("Listening...")
2632     if not LOCAL_MODE:
2633         # brief cyan pulse on the face LEDs so the user can see the robot is actively
           ↪ listening; ears also go green
2634         nao_set_leds(ssh, "FaceLeds", 0x0000FFFF, 0.2)
2635         nao_set_leds(ssh, "EarLeds", 0x0000FF00, 0.3)
2636
2637         # adaptive think-budget based on previous round's state + current face
2638         prev_state = (engine.history[-1].inferred_state if engine.history
2639                      else InferredState.COMFORTABLE)
2640         prev_rt = engine.history[-1].response_time if engine.history else 0.0
2641         no_speech_max, silence_secs, record_max_secs = engine.recommend_think_budget(
2642             state=prev_state, expression=expression,
2643             prev_response_time=prev_rt,
2644             consecutive_silences=engine.consecutive_silences,
2645             waiting=game_state.waiting,
2646         )
2647         dashboard.update_think_budget(record_max_secs)
2648
2649         record(ssh, energy_threshold,
2650               no_speech_max=no_speech_max,
2651               silence_secs=silence_secs,
2652               record_max_secs=record_max_secs)
2653         response_time = time.time() - question_start
2654
2655         # extended think-budget consumed; reset for next turn
2656         if game_state.waiting:
2657             game_state.waiting = False
2658
2659         vocal_emo, vocal_conf = classify_speech_emotion(speech_model, LOCAL_WAV)
2660         if vocal_conf < VOICE_CONFIDENCE_THRESHOLD:
2661             print(f" Vocal emotion: {vocal_emo} ({vocal_conf:.2f}): LOW CONFIDENCE, treating as
2662                   ↪ neutral")
2663             vocal_emo = "neutral"
2664         else:
2665             print(f" Vocal emotion: {vocal_emo} ({vocal_conf:.2f})")
2666
2667         vol_rms = measure_volume()
2668         print(f" Volume RMS: {vol_rms:.0f}")
2669
2670         # (c) transcribe; per-chunk gate, file-wide RMS removed
2671         relisten = lambda: record(ssh, energy_threshold,
2672                                   no_speech_max=no_speech_max,
2673                                   silence_secs=silence_secs,
2674                                   record_max_secs=record_max_secs)
2675         if INPUT_IS_LOCAL: # hybrid check
2676             if _local_speech_detected:
2677                 user_text = transcribe(bypass_wake_word=True, record_again=relisten)
2678             else:
2679                 print(f" No speech chunk detected (floor={LOCAL_SILENCE_RMS}); skipping
2680                       ↪ transcription.")
2681                 user_text = ""

```

```

2680 elif vol_rms >= NAO_MIN_RMS_TO_TRANSCRIBE:
2681     print(f" NAO RMS diagnostic: {vol_rms:.0f} (gate floor {NAO_MIN_RMS_TO_TRANSCRIBE})")
2682     user_text = transcribe(bypass_wake_word=True, record_again=relisten)
2683 else:
2684     print(f" WAV near-empty ({vol_rms:.0f} < {NAO_MIN_RMS_TO_TRANSCRIBE}); skipping
        ↪ transcription.")
2685     user_text = ""
2686
2687 if user_text:
2688     print(f" Heard: {user_text}")
2689     dashboard.append_user_speech(user_text)
2690     # user spoke; reset silence + nudge so future silence re-arms from scratch
2691     engine.consecutive_silences = 0
2692     nudge_level = 0
2693
2694     if is_goodbye(user_text):
2695         dashboard.quit_app()
2696 else:
2697     print(" Heard: (silence)")
2698     dashboard.append_user_speech("(silence)")
2699     engine.consecutive_silences += 1
2700
2701     # surface signals to dashboard even though the LLM is skipped this turn
2702     dashboard.update_signals(
2703         round_num=turn_count, user_answer="", correct_answer="",
2704         correct=False, expression=expression, expr_conf=expr_conf,
2705         vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2706         vol_rms=vol_rms, response_time=response_time,
2707         rolling_acc=engine.rolling_correctness(),
2708         total_correct=engine.total_correct,
2709         total_rounds=engine.total_rounds_played,
2710         streak=engine.consecutive_correct,
2711     )
2712
2713     # 1- check-in at 3 silences (proposal: "intervenes to re-engage them")
2714     if engine.consecutive_silences >= 3 and nudge_level < 1:
2715         nudge_prompt = [
2716             {"role": "system",
2717              "content": BASE_SYSTEM_PROMPT},
2718             {"role": "user",
2719              "content": "[The user has been quiet for 3 turns. Offer ONE brief, gentle
        ↪ check-in; no question, no nagging. 1 sentence max.]"},
2720         ]
2721         nudge_msg = converse(nudge_prompt, [])
2722         nudge_text = (nudge_msg.content or "").strip()
2723         nudge_speech = re.sub(r'\[gesture:\w+\]', '', nudge_text).strip()
2724         if not LOCAL_MODE:
2725             # recolour eyes to signal the state has shifted; user sees the shift even if
        ↪ they say nothing back
2726             nao_set_leds(ssh, "FaceLeds",
2727                          LED_COLOURS[InferredState.DISENGAGED], 0.4)
2728         if nudge_speech:
2729             say(ssh_tts, nudge_speech)
2730             print(f"\nRobot (nudge): {nudge_speech}")
2731             dashboard.update_robot_speech(nudge_speech)
2732             nudge_level = 1
2733
2734     # 2- switch game, soft re-invitation
2735     elif engine.consecutive_silences >= 4 and nudge_level < 2:

```

```

2736     engine.current_game = engine.pick_different_game()
2737     engine.game_switch_count += 1
2738     escalate_prompt = [
2739         {"role": "system",
2740          "content": BASE_SYSTEM_PROMPT},
2741         {"role": "user",
2742          "content": f"[The user has gone silent for 4 turns. Warmly acknowledge
                ↪ they've gone quiet then offer a {engine.current_game.value} round as a
                ↪ soft alternative. Two sentences total.]"},
2743     ]
2744     esc_msg = converse(escalate_prompt, [])
2745     esc_text = (esc_msg.content or "").strip()
2746     esc_speech = re.sub(r'\[gesture:\w+\]', '', esc_text).strip()
2747     if not LOCAL_MODE:
2748         nao_set_leds(ssh, "FaceLeds",
2749                     LED_COLOURS[InferredState.DISENGAGED], 0.4)
2750     if esc_speech:
2751         say(ssh_tts, esc_speech)
2752         print(f"\nRobot (escalate): {esc_speech}")
2753         dashboard.update_robot_speech(esc_speech)
2754     nudge_level = 2
2755
2756     continue # skip the LLM call; just wait for the user
2757
2758     if user_text.lower().strip() in [
2759         "stop", "quit", "exit", "goodbye", "bye", "end",
2760         "i want to stop", "let's stop", "no more",
2761     ]:
2762         print("Patient wants to stop.")
2763         break
2764
2765     signal_ctx = build_signal_context(
2766         engine, expression, expr_conf,
2767         vocal_emo, vocal_conf, vol_rms, response_time,
2768     )
2769
2770     user_msg_content = f"{signal_ctx}\n\nUser says: {user_text}"
2771     conversation.append({"role": "user", "content": user_msg_content})
2772
2773     # trim conversation to prevent context overflow; keep system + last 40 messages (20
2774         ↪ exchanges)
2775     if len(conversation) > 42:
2776         conversation = [conversation[0]] + conversation[-40:]
2777
2778     if not LOCAL_MODE:
2779         nao_set_leds(ssh, "EarLeds", 0x000000FF, 0.3) # blue = thinking
2780
2781     llm_message = converse(conversation, TOOLS)
2782
2783     response_text = process_llm_response(
2784         llm_message, conversation, engine, game_state,
2785         preferred_game, dashboard
2786     )
2787
2788     if not response_text.strip():
2789         response_text = "I'm here! What would you like to talk about? [gesture:neutral]"
2790
2791     gesture_type = extract_gesture(response_text)
2792     speech_text = re.sub(r'\[gesture:\w+\]', '', response_text).strip()

```

```

2792
2793     if not LOCAL_MODE:
2794         if engine.adaptation_log:
2795             last_state_str = engine.adaptation_log[-1]["state"]
2796             try:
2797                 last_state = InferredState(last_state_str)
2798             except ValueError:
2799                 last_state = InferredState.COMFORTABLE
2800             nao_set_leds(
2801                 ssh, "FaceLeds",
2802                 LED_COLOURS.get(last_state, 0x00FFFFFF), 0.5
2803             )
2804
2805     say(ssh_tts, speech_text)
2806
2807     print(f"\nRobot: {speech_text}")
2808     if game_state.current_answer:
2809         print(f"(Game answer: {game_state.current_answer})")
2810     dashboard.update_robot_speech(speech_text)
2811
2812     # (k) update dashboard (signals + decision if game active); use the actual answer check
2813         ↪ result from the tool chain
2814     was_game_answer = game_state.last_answer_checked
2815     correct = game_state.last_answer_correct if was_game_answer else False
2816
2817     if not LOCAL_MODE and was_game_answer and correct:
2818         threading.Thread(target=nao_gesture, args=(ssh, "correct_wave"), daemon=True).start()
2819
2820     # only run engine.decide on real game answers; chat or more-time turns mustn't ramp
2821         ↪ difficulty or pollute streak counters
2822     if was_game_answer:
2823         # use snapshot so a same-chain generate_game_question can't pollute this round's
2824             ↪ logged values
2825         answered_q = game_state.answered_question or game_state.current_question
2826         answered_a = game_state.answered_answer or game_state.current_answer
2827         answered_gt = game_state.answered_game_type or engine.current_game
2828         answered_df = game_state.answered_difficulty or engine.current_difficulty
2829         decision = engine.decide(
2830             expression, expr_conf, response_time,
2831             correct=correct,
2832             answer_text=user_text,
2833             vocal_emotion=vocal_emo, vocal_conf=vocal_conf,
2834             volume_rms=vol_rms,
2835         )
2836         engine.record_round(RoundResult(
2837             round_number=turn_count,
2838             game_type=answered_gt,
2839             difficulty=answered_df,
2840             question=answered_q,
2841             user_answer=user_text,
2842             correct=correct,
2843             response_time=response_time,
2844             facial_expression=expression,
2845             expression_confidence=expr_conf,
2846             vocal_emotion=vocal_emo,
2847             vocal_emotion_confidence=vocal_conf,
2848             volume_rms=vol_rms,
2849             inferred_state=decision.inferred_state,
2850         ))

```

```

2848         # save after every round; power-cycle loses one turn, not five
2849         save_session(engine, preferred_game, quiet=True)
2850         dashboard.update_signals(
2851             round_num=turn_count, user_answer=user_text,
2852             correct_answer=answered_a,
2853             correct=correct,
2854             expression=expression, expr_conf=expr_conf,
2855             vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2856             vol_rms=vol_rms, response_time=response_time,
2857             rolling_acc=engine.rolling_correctness(),
2858             total_correct=engine.total_correct,
2859             total_rounds=engine.total_rounds_played,
2860             streak=engine.consecutive_correct,
2861         )
2862         dashboard.update_decision(decision)
2863         # clear snapshot now the round is logged
2864         game_state.answered_question = ""
2865         game_state.answered_answer = ""
2866         game_state.answered_game_type = None
2867         game_state.answered_difficulty = None
2868     else:
2869         # chat or more-time turn; refresh dashboard signals only
2870         dashboard.update_signals(
2871             round_num=turn_count, user_answer=user_text,
2872             correct_answer="",
2873             correct=False,
2874             expression=expression, expr_conf=expr_conf,
2875             vocal_emo=vocal_emo, vocal_conf=vocal_conf,
2876             vol_rms=vol_rms, response_time=response_time,
2877             rolling_acc=engine.rolling_correctness(),
2878             total_correct=engine.total_correct,
2879             total_rounds=engine.total_rounds_played,
2880             streak=engine.consecutive_correct,
2881         )
2882
2883     game_state.last_answer_checked = False
2884
2885     # auto-save every 2 turns
2886     if turn_count % 2 == 0:
2887         save_session(engine, preferred_game, quiet=True)
2888         print(" [Auto-save]")
2889
2890 except KeyboardInterrupt:
2891     print("\n\nInterrupted.")
2892
2893
2894 save_session(engine, preferred_game)
2895
2896 summary = engine.get_session_summary()
2897 print(f"\n{'=' * 60}")
2898 print(" SESSION SUMMARY")
2899 print(f"{'=' * 60}")
2900 for k, v in summary.items():
2901     print(f" {k}: {v}")
2902
2903 if summary.get("rounds", 0) > 0:
2904     acc = summary["accuracy"]
2905     streak_note = (f" Your best streak was {summary['best_streak']} in a row!"
2906                   if summary["best_streak"] >= 3 else "")

```

```

2907         if acc >= 0.8:
2908             farewell = f"Amazing session! You got {summary['correct']} out of {summary['rounds']}
                ↳ right.{streak_note} Brilliant work! Your progress is saved; see you next time!"
2909         elif acc >= 0.5:
2910             farewell = f"Great effort! You scored {summary['correct']} out of
                ↳ {summary['rounds']}.{streak_note} Well played! Your progress is saved; see you
                ↳ next time!"
2911         else:
2912             farewell = f"Thanks for playing! You got {summary['correct']} out of
                ↳ {summary['rounds']}.{streak_note} Every round is a learning opportunity. Your
                ↳ progress is saved; see you next time!"
2913     else:
2914         farewell = "It was great to chat! Your progress is saved; see you next time!"
2915
2916     if not LOCAL_MODE:
2917         nao_gesture(ssh, "wave")
2918     say(ssh_tts, farewell)
2919     print(f"\nRobot: {farewell}")
2920
2921     if local_camera is not None:
2922         local_camera.release()
2923     if not LOCAL_MODE:
2924         nao_track_face(ssh, enable=False)
2925         nao_set_leds(ssh, "FaceLeds", 0x00000000, 0.5)
2926         ssh.close()
2927         ssh_tts.close()
2928     dashboard.close()
2929     print("\nGAZE disconnected.")
2930
2931     # ----- MAIN ENTRY-POINT -----
2932
2933     def main():
2934         print("-----")
2935         print(" GAZE: Game-Adaptive Zone of Engagement")
2936         print(" Adaptive Game-System Ran on Pepper")
2937         print("-----")
2938
2939         print("\nLoading facial expression model...")
2940         face_model = FacialExpressionModel(MODEL_JSON, MODEL_WEIGHTS)
2941         face_cascade = cv2.CascadeClassifier(HAAR_CASCADE)
2942         print(" Facial model loaded.")
2943
2944         speech_model = None
2945         if os.path.exists(SPEECH_MODEL):
2946             print("Loading speech emotion model...")
2947             speech_model = SpeechEmotionModel(SPEECH_MODEL)
2948             print(" Speech model loaded.")
2949         else:
2950             print(f" Speech emotion model not found at {SPEECH_MODEL}; vocal signal disabled.")
2951             print(" Run train_speech_model.py to generate it.")
2952
2953         local_camera = None
2954         if USE_LOCAL_CAMERA:
2955             local_camera = cv2.VideoCapture(0)
2956             # BUFFER_SIZE=1; else OpenCV buffers stale frames
2957             local_camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
2958             local_camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
2959             local_camera.set(cv2.CAP_PROP_FPS, 30)
2960             local_camera.set(cv2.CAP_PROP_BUFFERSIZE, 1)

```



```

2961     print(" Using local webcam for expression detection.")
2962     if DEBUG_PREVIEW:
2963         start_preview_thread(local_camera, face_model, face_cascade)
2964         print(" Preview thread started.")
2965     else:
2966         print(" Using Pepper's camera for expression detection.")
2967
2968     # connect to Pepper (skipped in local mode)
2969     ssh = None
2970     ssh_tts = None
2971     energy_threshold = DEFAULT_ENERGY_THRESHOLD
2972
2973     # Pepper SSH connection: required whenever output goes through the robot
2974     # (TTS, LEDs, gestures, face-tracking). LOCAL_MODE remains the gate for that.
2975     if LOCAL_MODE:
2976         print("\n LOCAL MODE: skipping Pepper connection.")
2977     else:
2978         print(f"\nConnecting to Pepper at {NAO_IP}...")
2979         ssh = ssh_connect()
2980         ssh_tts = ssh_connect()    # dedicated TTS connection
2981         print(" Connected.")
2982
2983     # Pepper stream ~10 fps; detect at 6-7 Hz
2984     if not USE_LOCAL_CAMERA:
2985         print(" Starting Pepper camera stream...")
2986         threading.Thread(target=pepper_video_loop,
2987                         args=(ssh,), daemon=True).start()
2988         threading.Thread(target=pepper_camera_receive_loop,
2989                         args=(NAO_IP,), daemon=True).start()
2990         threading.Thread(target=detect_thread_loop,
2991                         args=(face_model, face_cascade), daemon=True).start()
2992         print(" Pepper camera stream threads started.")
2993
2994     # Mic-side ambient calibration: pick whichever microphone INPUT_IS_LOCAL says.
2995     # In gaze21 hybrid mode this is the Mac mic even though Pepper is connected.
2996     if INPUT_IS_LOCAL:
2997         print("\nCalibrating Mac microphone ambient noise level...")
2998         local_calibrate_ambient()
2999     else:
3000         print("\nCalibrating ambient noise level (stay quiet for 3 seconds)...")
3001         energy_threshold = nao_calibrate_ambient(ssh)
3002
3003     dashboard = GazeDashboard(ssh=ssh, ssh_tts=ssh_tts)
3004     print(" Dashboard launched.")
3005
3006     conv_thread = threading.Thread(
3007         target=conversation_loop,
3008         args=(dashboard, face_model, face_cascade, speech_model,
3009             local_camera, ssh, ssh_tts, energy_threshold),
3010         daemon=True,
3011     )
3012     conv_thread.start()
3013
3014     # tkinter mainloop on main thread (macOS-required); keep camera preview and GUI responsive
3015     ↳ whilst the conversation loop blocks on I/O
3016     dashboard.root.mainloop()
3017 if __name__ == "__main__":
3018     main()

```